



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda

ALGORITMOS ED - LINGUAGEM ALGORÍTMICA

Engenharia Informática, 2013/2014, A1, S1

PAULO NUNES

AV. DR. FRANCISCO SÁ CARNEIRO, 50 - 6301-559 GUARDA

TELF. 271220161, EXT. 161, GAB:20

GPS: LATITUDE: 40.5416236730513, LONGITUDE: -7.28243350982666

VOIP: pnunes@ipg.pt, MSN: pnunes@ipg.pt, SKYPE: pnunes.ipg.pt

EMAIL: Mailto:pnunes@ipg.pt, WEB: <http://www.ipg.pt/user/~pnunes/>



CONTEÚDOS PROGRAMÁTICOS

3. Linguagem algorítmica

1. Conceito de algoritmo
2. Fases de desenvolvimento de um algoritmo
3. Importância, sintaxe
4. Fluxograma e pseudocódigo
5. Tipos de variáveis simples, tipos de variáveis estruturados
6. Instruções de entrada e saída
7. Estruturas de controlo
 1. Decisão
 2. Repetição



INTRODUÇÃO

- ❑ Para resolver um problema através dum computador é necessário, em primeiro lugar, descrevê-lo de uma forma clara e precisa.
- ❑ É também necessário que encontremos os passos necessários para a sua resolução.
- ❑ Ou seja, é necessário encontrar o algoritmo que descreva a resolução do problema.



INTRODUÇÃO

- ❑ Para os programadores a noção de **algoritmo** é fundamental.
 - ❑ Assim como uma **receita** para um cozinheiro,
 - ❑ uma **planta** para um construtor .

- ❑ Desenvolvimento de algoritmos é uma tarefa:
 - ❑ Muito difícil
 - ❑ É uma das maiores dificuldades para os programadores.
 - ❑ Extremamente aliciante
 - ❑ É um dos maiores desafios para os programadores.



ALGORITMO

❑ Dicionário da Língua Portuguesa

❑ <http://www.portoeditora.pt/dol/>

❑ substantivo masculino

MATEMÁTICA método e anotação das diversas operações e processos de calcular;

❑ programa que se introduz numa máquina de calcular;



ALGORITMO

□ Dicionário da Língua Portuguesa

- <http://www.portoeditora.pt/dol/>
 - substantivo masculino

INFORMÁTICA

neologismo conjunto de regras que hão-de levar à solução de um problema;

- conjunto de fases de pormenor por que é preciso passar para chegar à solução de um problema;

- (Do lat. med. *algorithmu*-, «algoritmo», pelo ing. *algorithm*, «id.»)



ALGORITMO

- ❑ É uma sequência de passos para realizar uma tarefa ou resolver um problema.

- ❑ As pessoas no seu dia a dia utilizam **dezenas de algoritmos** para realizar as suas actividades.
 - ❑ Cada pessoa, desde que acorda até voltar a adormecer novamente, define uma sequência de actividades que tem de realizar para atingir os seus objectivos.



EXEMPLO DE ALGORITMOS

- ❑ As indicações fornecidas a uma pessoa descrevendo como ir para um dado local
 - ❑ são um algoritmo para encontrar esse local.

- ❑ Uma receita culinária é também um algoritmo, descreve a sequência de passos necessária para obter um dado cozinhado.



ALGORITMOS

1. Como chegar ao Vivaci.
2. Comprar um bilhete de cinema.
3. Levantar dinheiro no multibanco.
4. Trocar um pneu de um automóvel.
5. Colocar combustível no automóvel.
6. Telefonar para a PSP da Guarda.
7. Ir para a sala 21 da ESTG.



ALGORITMOS

8. Utilizar o Excel para converter um valor decimal em Binário.
9. Visualizar a página Web do IPG.
10. Como entrar nos computadores da sala.
11. Utilizando o Word, inserir números de página num documento.
12. Fazer circular a folha de presenças.



ESCREVER O ALGORITMO

13. Carregar o telemóvel no multibanco.
14. Como ir para as urgências do Hospital da Guarda.
15. Quais os jogos da próxima jornada do campeonato Português de Futebol.
16. Como lanchar no Bar da ESTG



ALGORITMO - CARACTERÍSTICAS

❑ Finitude

- ❑ Um algoritmo tem de terminar ao fim de um número finito de passos.

❑ Definitude

- ❑ Cada passo do algoritmo tem de ser definido com precisão.

❑ Entrada

- ❑ Um algoritmo pode ter zero ou mais entradas.

❑ Saídas

- ❑ Um algoritmo tem uma ou mais saídas.

❑ Eficácia

- ❑ Todas as operações feitas por um algoritmo têm de ser básicas.



FASES DESENV. ALGORITMOS

1. Definição do problema
2. Obtenção e desenvolvimento de um modelo
3. Desenho ou esboço do algoritmo
4. Precisão do algoritmo
5. Análise de complexidade do algoritmo
6. Implementação
7. Teste do programa
8. Documentação



DEFINIÇÃO DO PROBLEMA

- ❑ Antes de começar a escrever o algoritmo, o problema tem de ser bem especificado, sob pena do algoritmo ter de ser desenvolvido com tempos e esforços muito maiores.

RESOLUÇÃO DE PROBLEMAS

- ❑ O que é um problema ?
 - ❑ Questão que se propõe ser resolvida, enigma, dúvida.
 - ❑ Qualquer situação onde existe uma diferença entre o que existe na realidade e o que se deseja.





PROBLEMAS DIFEREM EM GRAU DE ESTRUTURA

1. Bem estruturados

- ❑ Está disponível toda a informação;
- ❑ Utiliza-se um procedimento padrão para obter a solução (Algoritmo).

2. Semi-estruturados

- ❑ Apenas se pode definir o problema parcialmente devido à incerteza sobre o estado inicial, o estado final, ou de qualquer outra informação;
- ❑ requer a combinação de um procedimento padrão e de uma resposta criativa (heurísticos).



PROBLEMAS DIFEREM EM GRAU DE ESTRUTURA

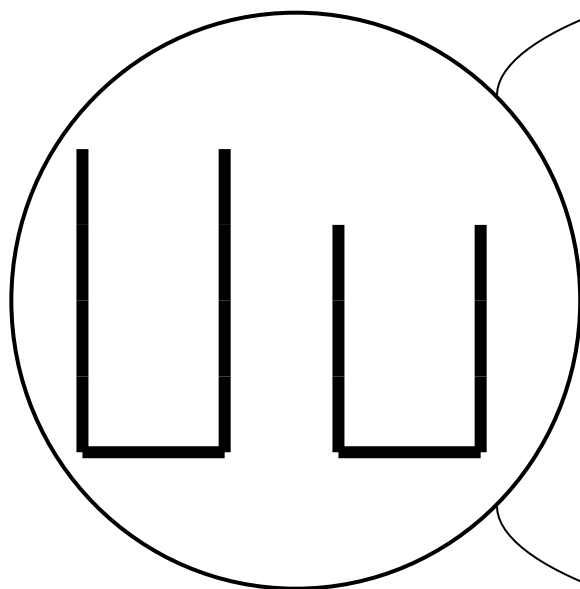
3. Mal estruturados

- ❑ Existe pouca ou nenhuma informação de como desenvolver uma solução;
- ❑ A informação para resolver o problema deve ser gerada durante o processo de resolução do problema - a solução é improvisada usando aproximação heurísticas.

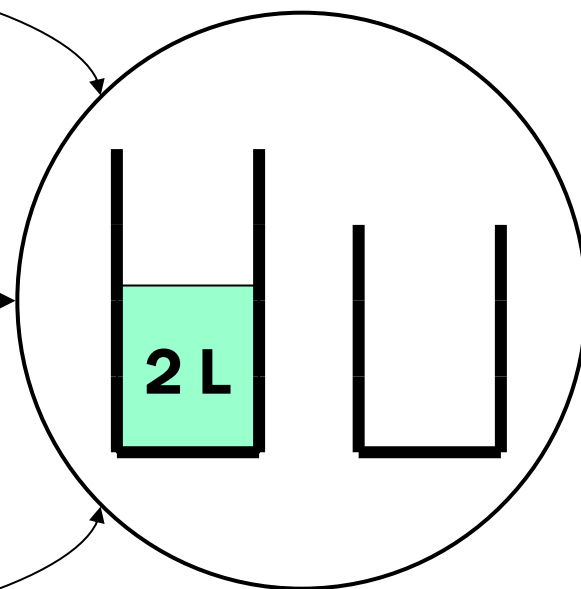


PROBLEMA DO VASILHAME

Estado Inicial



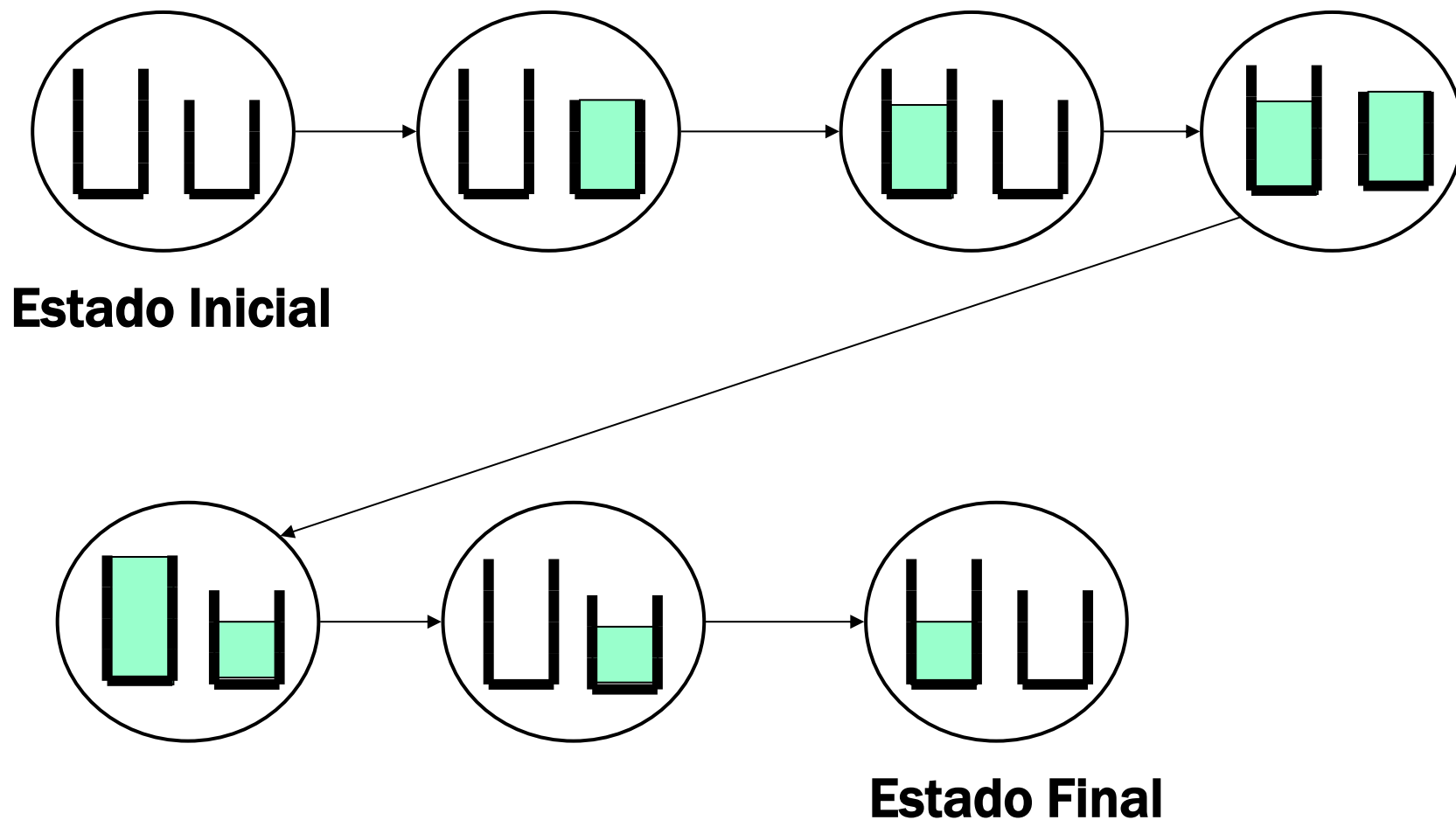
Estado Final



Vasilhas de 4 e 3 Litros
Uma torneira para enchei

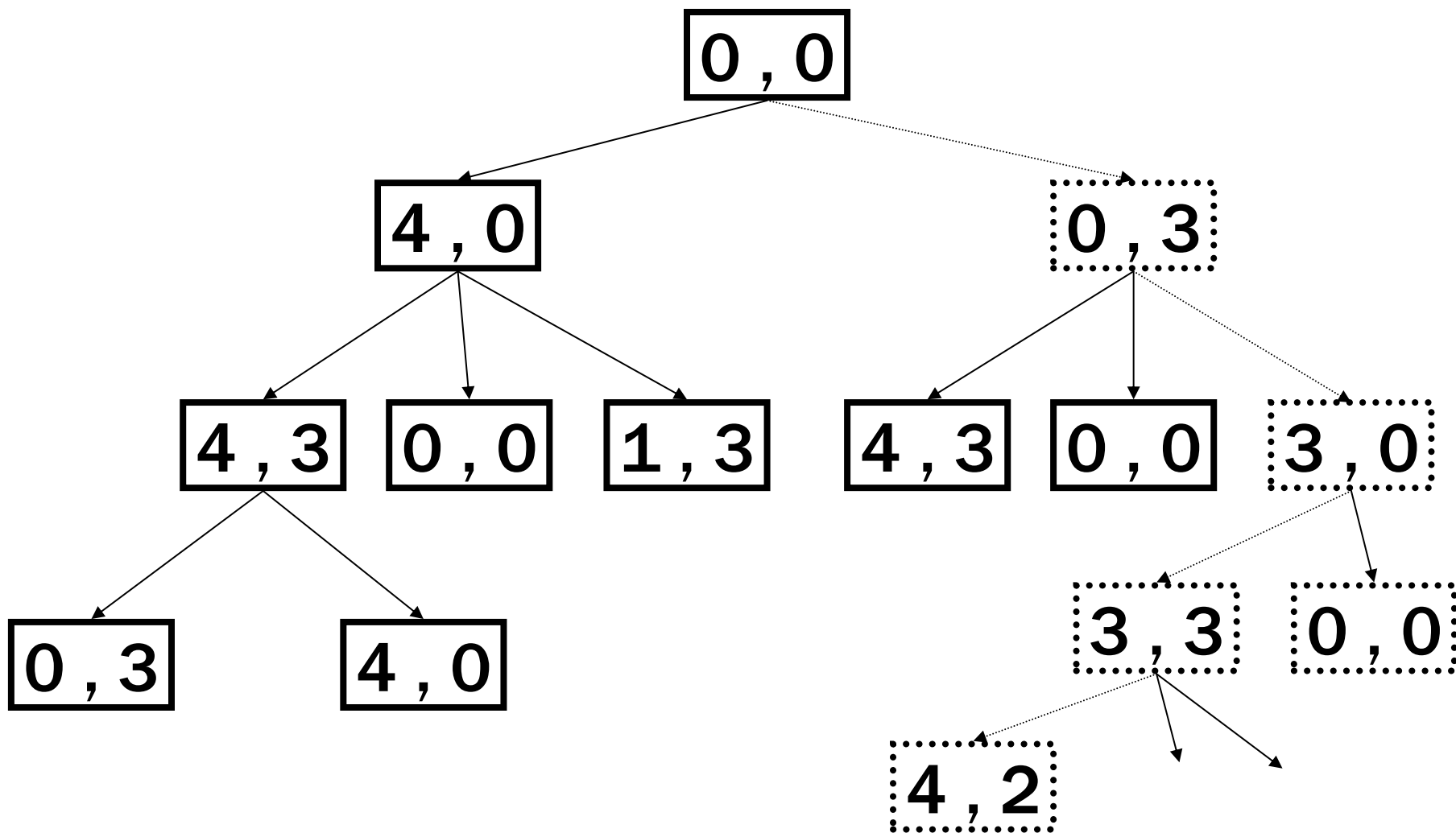


PROBLEMA DO VASILHAME





PROBLEMA DO VASILHAME





DESCRIÇÃO FORMAL DO PROBLEMA

1. Definir o espaço de estados que contenha todas as possíveis configurações dos objectos relevantes;
2. Especificar um ou mais estados iniciais possíveis;
3. Especificar um ou mais estados que seriam aceitáveis como soluções para o problema (estados meta);
4. Especificar um conjunto de regras que descrevam as operações disponíveis:
 1. Que suposições não declaradas na descrição considerar?;
 2. Qual a generalidade a dar às regras?;
 3. Que trabalho deveria ser pré-calculado e representado nas regras?.



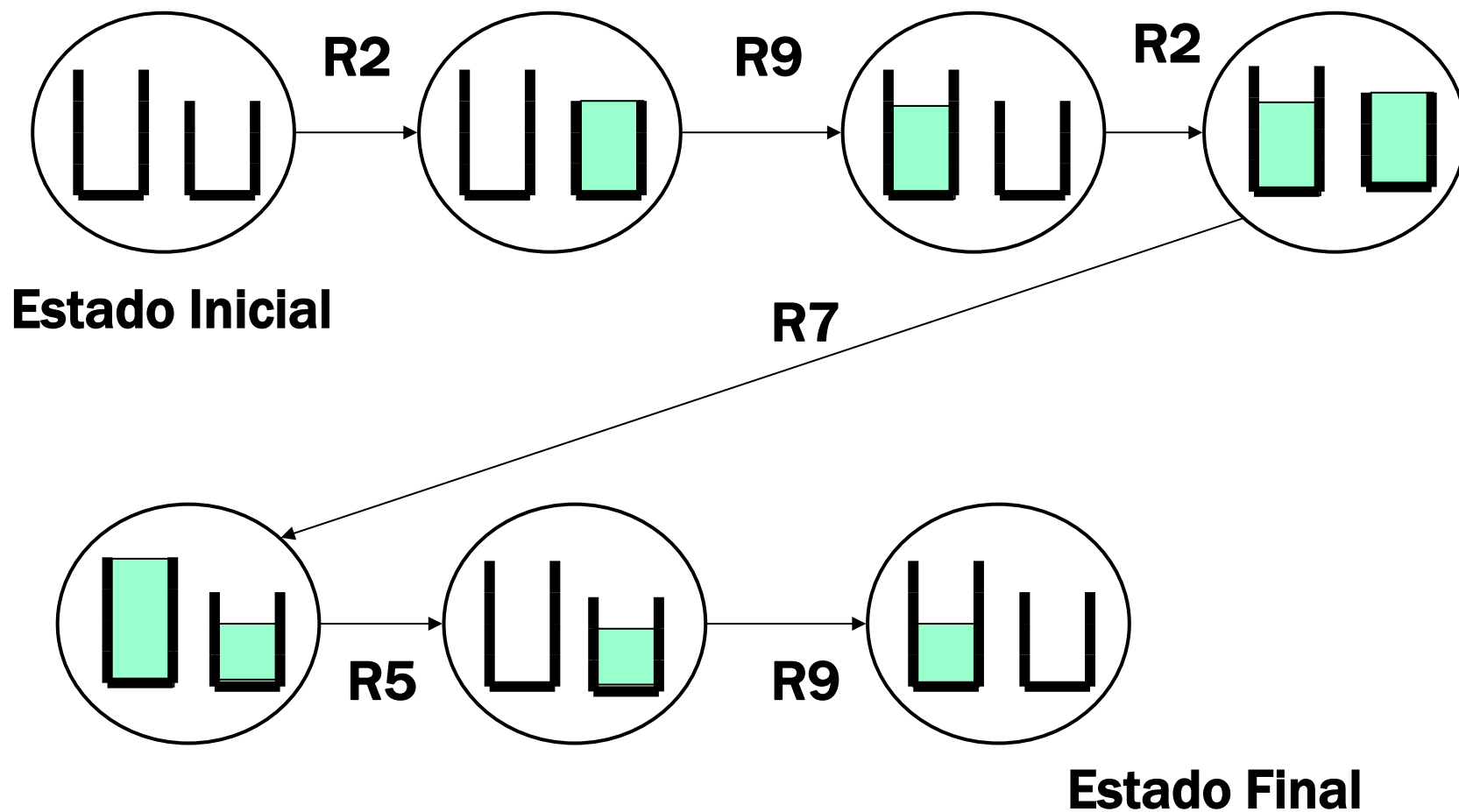
RESOLUÇÃO DO PROBLEMA

□ Regras + Estratégia de Controlo

- 1. Encher o vasilhame de 4 litros
- 2. Encher o vasilhame de 3 litros
- 3. Despejar alguma água do vasilhame de 4 litros
- 4. Despejar alguma água do vasilhame de 3 litros
- 5. Esvaziar o vasilhame de 4 litros
- 6. Esvaziar o vasilhame de 3 litros
- 7. Despejar água do vasilhame de 3 litros dentro do vasilhame de 4 litros até que o vasilhame de 4 litros esteja cheio
- 8. Despejar água do vasilhame de 4 litros dentro do vasilhame de 3 litros até que o vasilhame de 3 litros esteja cheio
- 9. Despejar toda a água do vasilhame de 3 litros no vasilhame de 4 litros
- 10. Despejar toda a água do vasilhame de 4 litros no vasilhame de 3 litros



PROBLEMA DO VASILHAME



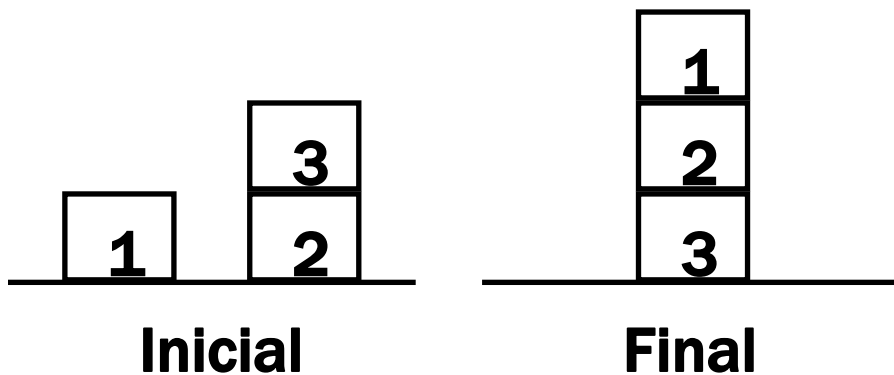


MÉTODOS DE PROCURA

- ❑ Que regra aplicar primeiro ?
 - ❑ 1. Causar Movimento.
 - ❑ 2. Ser Sistemática.
 - ❑ A) Escolher sempre a 1ª regra.
 - ❑ B) Escolher uma regra aleatoriamente.
 - ❑ C) Estado inicial raiz:
 - ❑ (a) gerar todos os filhos (possíveis)
 - ❑ (b) para cada filho repetir (a).
- ❑ Busca em amplitude (largura) e profundidade.



BLOCOS



Regra:

Mover X para cima de Y



BLOCOS

Canibais e Missionários

CCC MMM

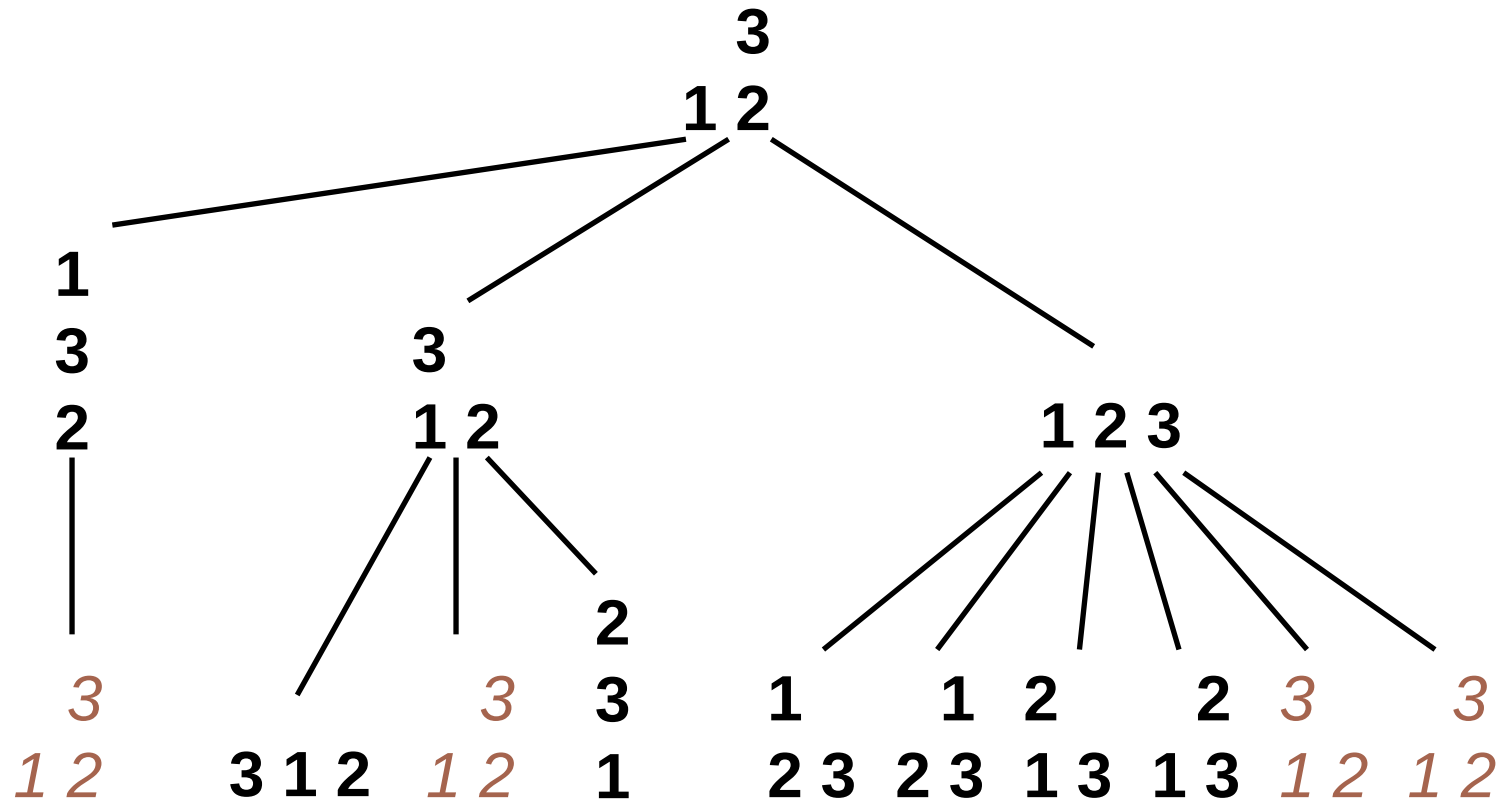
Rio

Rio

CCC MMM



Fila





EXERCÍCIOS

- ❑ Vasilhame
- ❑ Blocos
- ❑ Canibais e Missionários
- ❑ Torres de Hanói
- ❑ Puzzle-8
- ❑ Labirinto
- ❑ Macaco e Bananas
- ❑ Caixeiro Viajante



ALGORITMOS - ESTRUTURAS DADOS

- ❑ Algoritmos são métodos para resolver problemas:
 - ❑ problemas têm dados;
 - ❑ dados são tratados computacionalmente.



ALGORITMOS - ESTRUTURAS DADOS

- ❑ O Processo de organização dos dados pode determinar a eficiência do algoritmo:
 - ❑ algoritmos simples podem requerer estruturas de dados complexas
 - ❑ algoritmos complexos podem requerer estruturas de dados simples



ALGORITMOS - ESTRUTURAS DADOS

- ❑ Para maximizar a eficiência de um algoritmo as estruturas de dados que se usam têm de ser projectadas em simultâneo com o desenvolvimento do algoritmo.



PORQUÊ ESTUDAR ALGORITMOS?

- ❑ Quanto se utilizam meios computacionais:
 - ❑ queremos sempre que a computação seja mais rápida;
 - ❑ queremos ter a possibilidade de processar mais dados;
 - ❑ queremos algo que seria impossível sem o auxílio de um computador.



DEFINIÇÃO DO PROBLEMA

- ❑ Antes de começar a escrever o algoritmo, o problema tem de ser bem especificado, sob pena do algoritmo ter de ser desenvolvido com tempos e esforços muito maiores.



DESENHO OU ESBOÇO DO ALGORITMO

- ❑ É o “protótipo” do algoritmo. Pode ser utilizada uma linguagem própria:
 - ❑ pseudocódigo
 - ❑ fluxograma
 - ❑ com símbolos previamente definidos.



PSEUDOCÓDIGO

- ❑ Por pseudocódigo entende-se como um código de escrita, onde se utilizam um misto de palavras da nossa linguagem natural com palavras e notações típicas das linguagens de programação.



PSEUDOCÓDIGO

- ❑ O pseudocódigo tem-se revelado mais prático e mais vantajoso na estruturação do raciocínio e dos elementos (acções, dados) a incluir num programa.



PSEUDOCÓDIGO: EXEMPLO

Início:

Ler Numero1

Ler Numero2

Soma $\leftarrow$ Numero1 + Numero2

Media $\leftarrow$ Soma/2

Escrever Soma, Média

Fim.

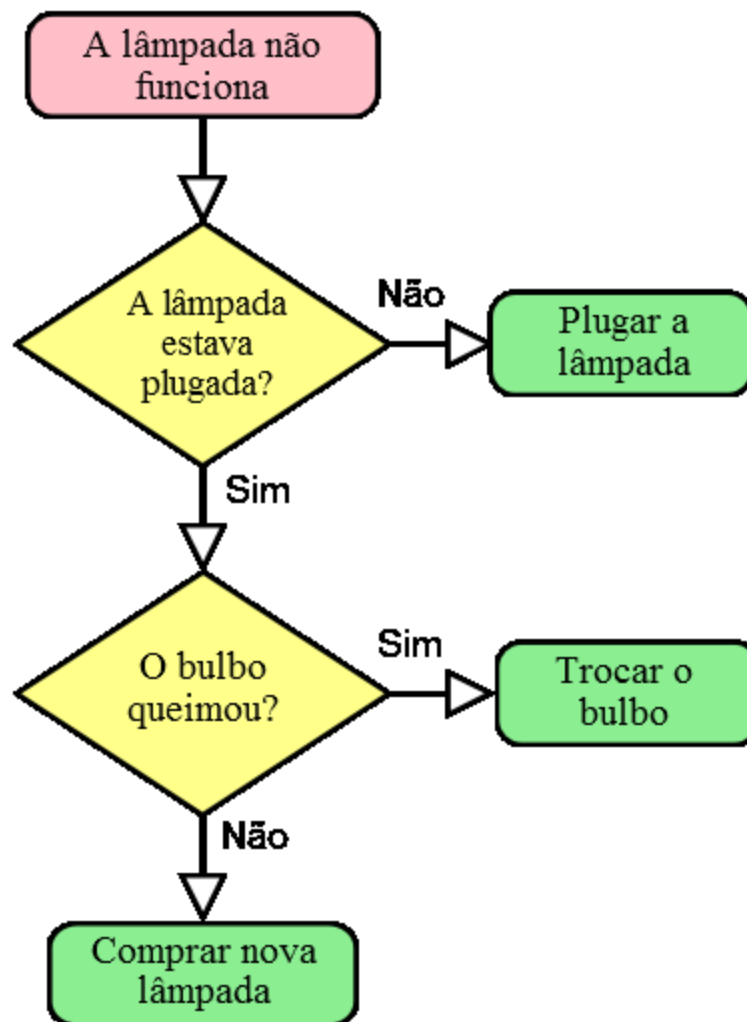


FLUXOGRAMA

- ❑ Os fluxogramas permitem a descrição de funções e de comportamentos, representando **passo-a-passo** a progressão utilizando linhas de ligação e um conjunto de símbolos.
- ❑ Os fluxogramas também ajudam a simplificar e sintetizar os processos/métodos realizados durante a execução de um trabalho.



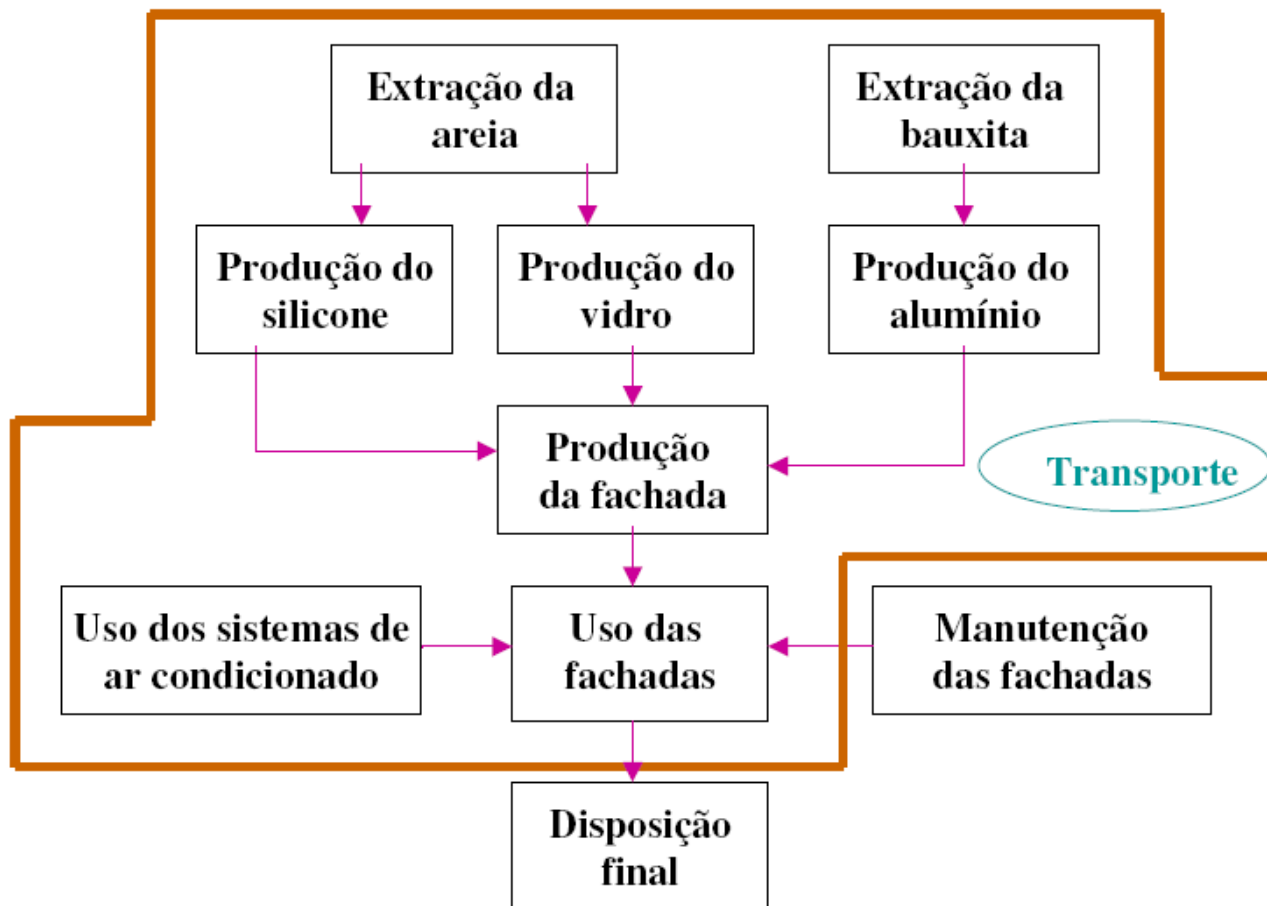
EXEMPLO: LÂMPADA



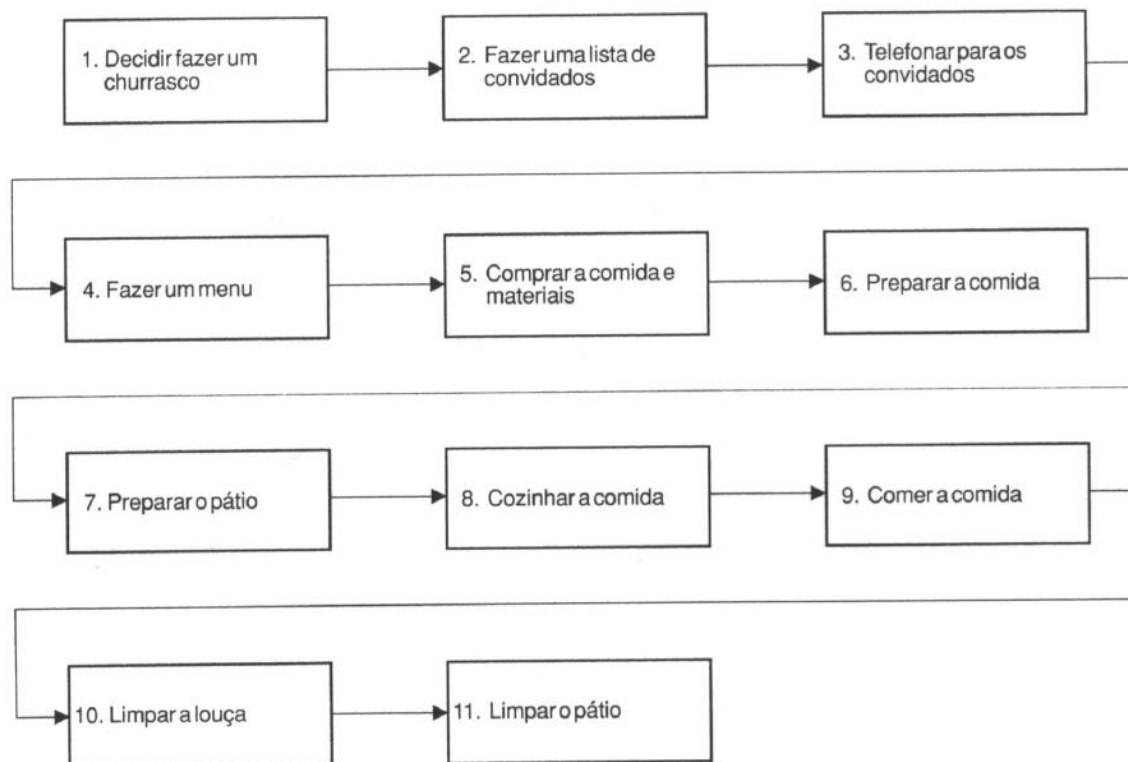
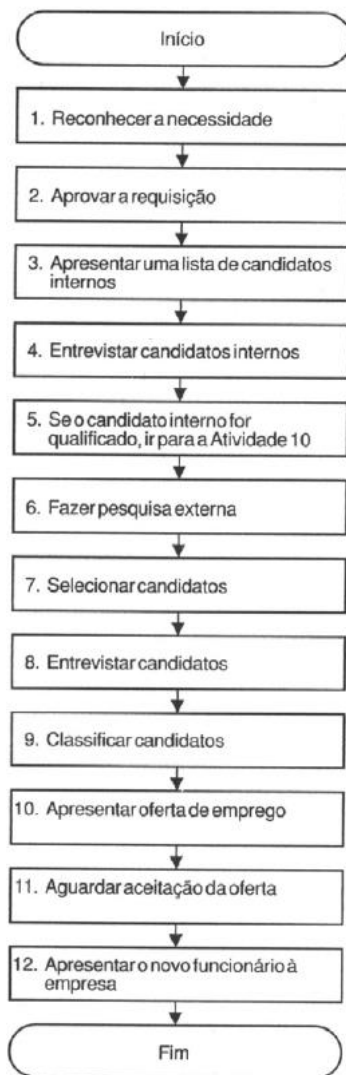
EXEMPLO

FRONTEIRAS DO SISTEMA

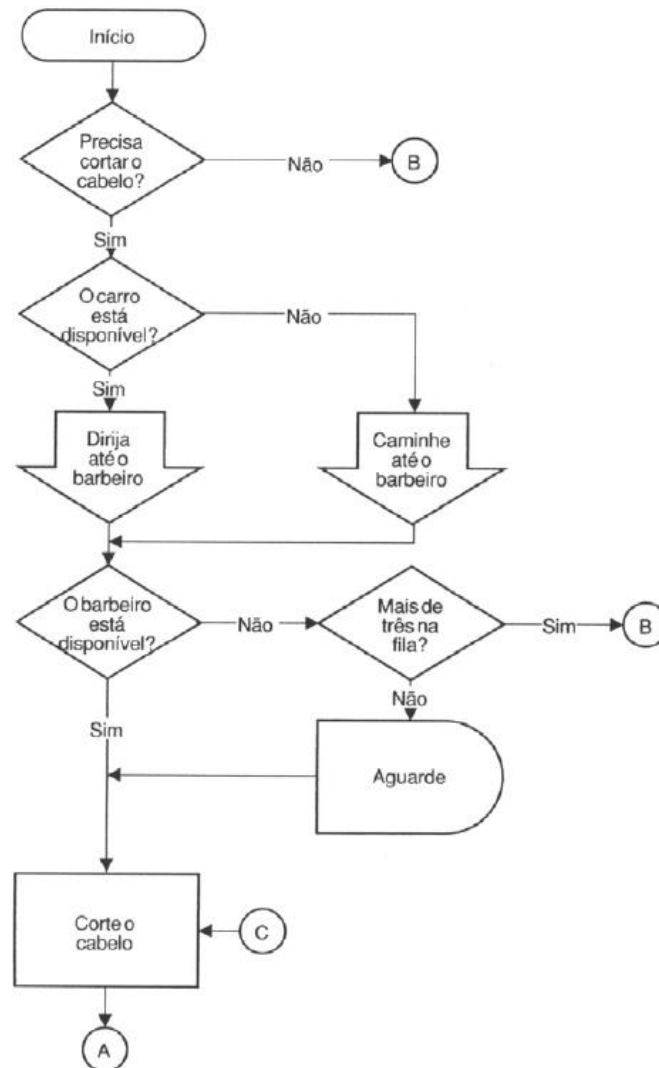
Exemplo: fachadas em structural glazing



EXEMPLOS



EXEMPLO: CORTAR CABELO





FLUXOGRAMAS

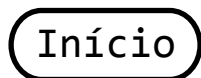
- ❑ Os fluxogramas são diagramas representativos do fluxo das acções de um programa, através de símbolos, que representam os diferentes tipos das acções e o seu encadeamento na sequência do programa.



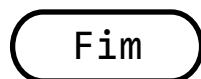
FLUXOGRAMAS

- ❑ Podem ser úteis para visualizar melhor determinadas estruturas de programação.
- ❑ Para representar programas de média ou grande extensão, tornam-se difíceis de concretizar – podem exigir muito espaço, para além do tamanho habitual de uma folha de papel, o que os torna de difícil leitura e manuseamento.

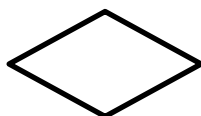
SÍMBOLOS



□ Início



□ Fim



□ Condição



□ Conector



□ Processamento



□ Saída para ecrã



□ Entrada do teclado



□ Sentido do fluxo

SÍMBOLOS: LISTA COMPLETA

	Armazenamento interno		Conector fora da pagina		Mesclar		Processo
	Documento		Cartão		Dados Armazenados		Processo alternativo
	Vários documentos		Fita perfurada		Atraso		Decisão
	Terminação		Somador		Armazenamento de acesso seqüencial		Dados
	Preparação		Ou		Disco magnético		Processo predefinido
	Entrada manual		Agrupar		Armazenamento de acesso direto		
	Operação manual		Classificar		Exibir		
	Conector		Extrair				



OPERADORES: ARITMÉTICOS

□ Números inteiros

- + Adição
- Subtracção
- / Divisão inteira
- × Multiplicação
- % Resto da divisão inteira
- ^ Potenciação



LÓGICOS

- E Conjunção
- OU Disjunção
- XOU Conjunção Exclusiva
- NAO Negação



OPERADORES: RELACIONAIS

☐ =	Igual
☐ <>	Diferente
☐ >	Maior
☐ >=	Maior ou igual
☐ <	Menor
☐ <=	Menor ou igual

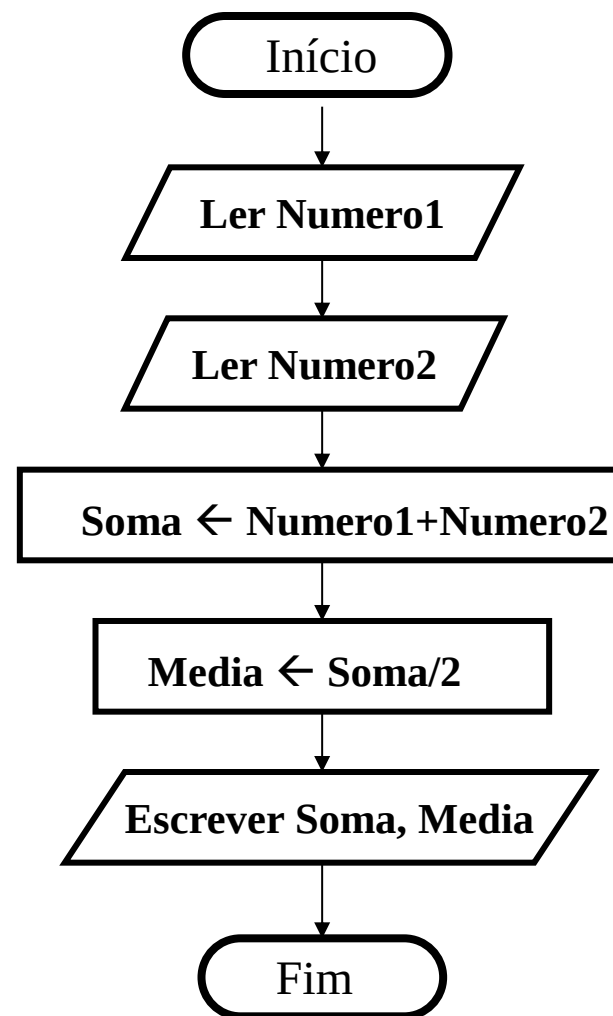
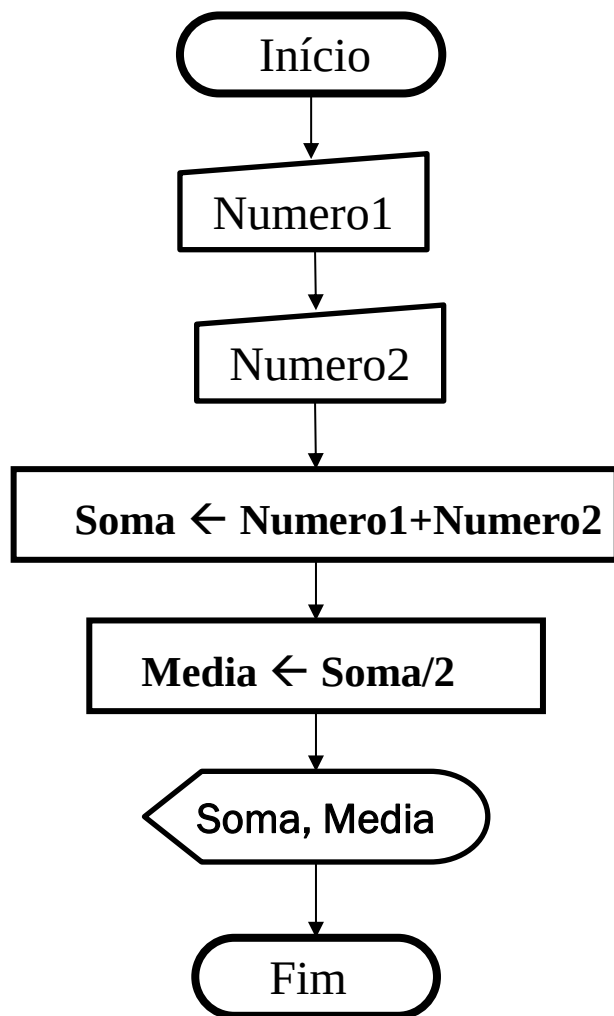


SOFTWARE PARA DESENHAR

- ❑ Microsoft Visio
- ❑ Dia (Necessita de: Bibliotecas GTK)
- ❑ Gerador de fluxogramas (Access + Visio)



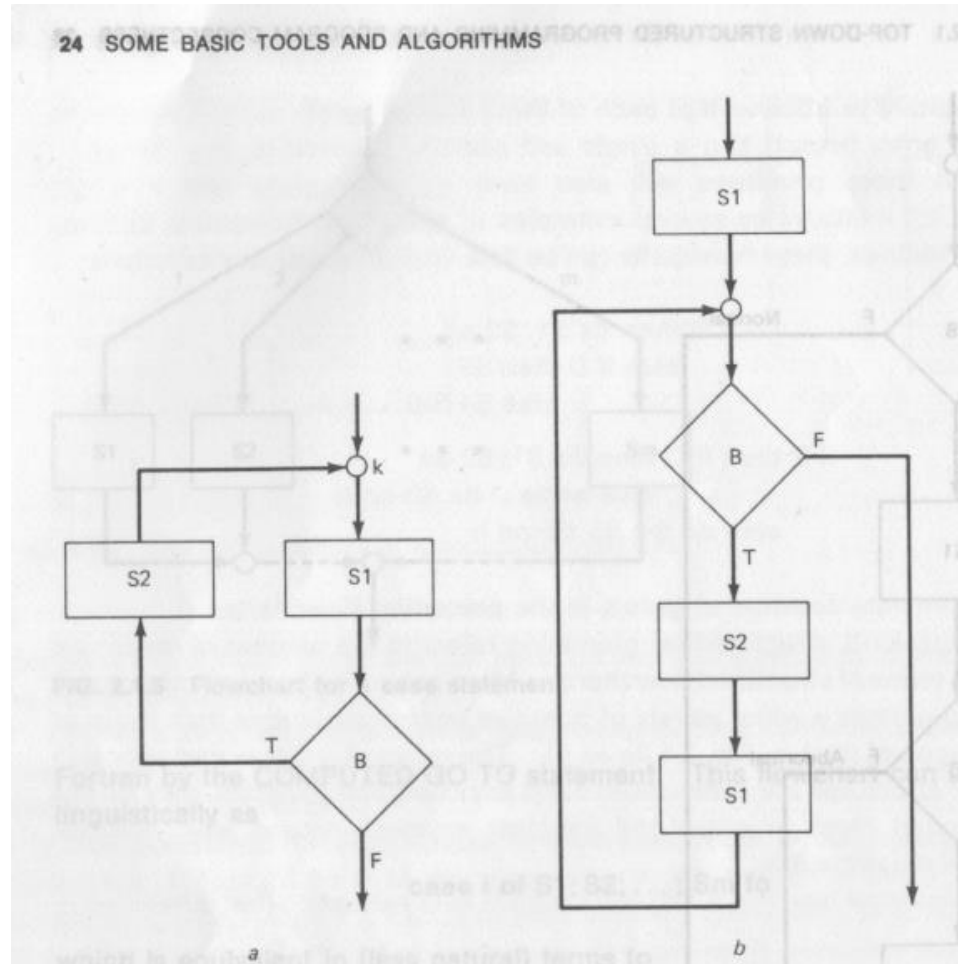
FLUXOGRAMA: EXEMPLO



FLUXOGRAMAS



Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda



FLUXOGRAMA

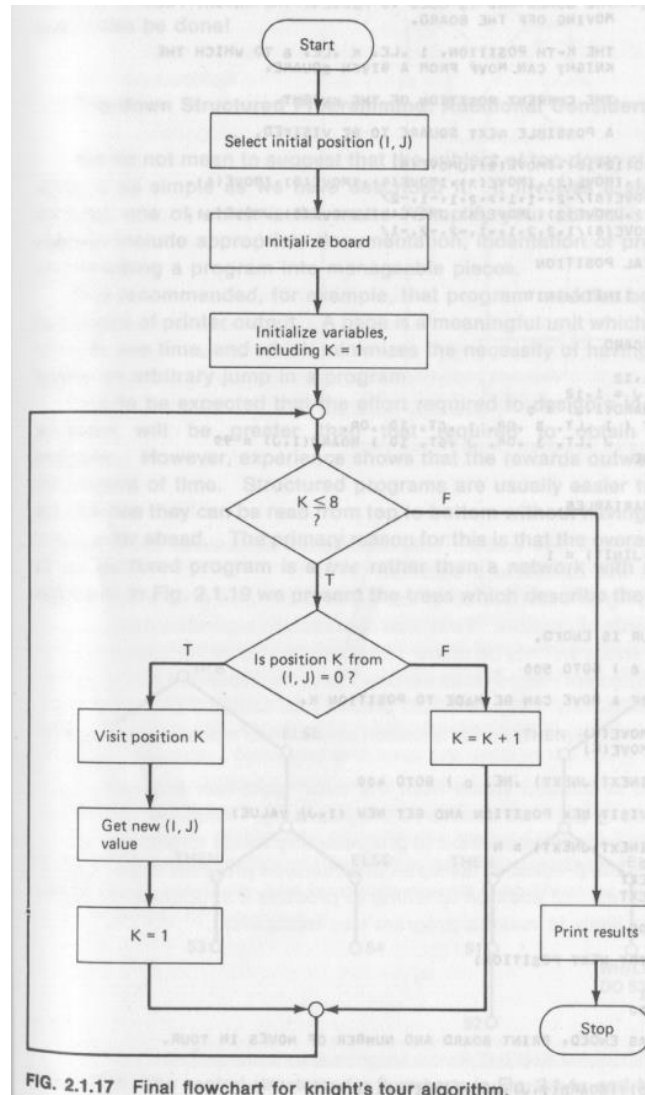


FIG. 2.1.17 Final flowchart for knight's tour algorithm.



DESENHO OU ESBOÇO DO ALGORITMO

Escola Superior de Tecnologia e Gestão
Instituto Politécnico da Guarda

Início:

Ler prova1

Ler prova2

Ler prova3

Ler prova4

Soma $\leftarrow$ prova1+prova2+prova3+prova4

MediaFinal $\leftarrow$ soma / 4

Escrever MediaFinal

Fim.



IMPORTÂNCIA, SINTAXE

- ❑ A linguagem algorítmica utiliza o português para a definição das instruções e tem as seguintes características:
 1. Começam por **início**: e terminam com **fim**.
 2. Cada linha contém apenas uma instrução
 3. O sinal de atribuição é a seta ($\leftarrow$)
 4. A definição de variáveis é realizada antes do algoritmo.

- ❑ Sintaxe
 - Início:
 - Algoritmo
 - Fim.



TIPOS DE VARIÁVEIS

- ❑ É um objecto capaz de reter e representar um valor.
 - ❑ Tem um identificador
 - ❑ Permite armazenar valores de um determinado domínio.

- ❑ Tipos
 - ❑ Simples
 - ❑ Permitem armazenar apenas um valor.
 - ❑ Números inteiros e reais, texto, etc.
 - ❑ Estruturados
 - ❑ Estruturas de dados compostas: data, hora, ficha de inscrição.
 - ❑ Permitem armazenar diversos valores de tipos diferentes.
 - ❑ O acesso é efectuado utilizando o ".".
 - ❑ Vectores e matrizes
 - ❑ Permitem armazenar diversos valores do mesmo tipo.
 - ❑ O acesso aos valores é baseado num endereço (número).

- ❑ Atribuição
 - ❑ ←
MediaFinal ← Soma / 4



Tipos de variáveis simples

- ❑ Nota – Nota a programação
 - ❑ Número inteiro.
 - ❑ Domínio: $[0, 20]$.
- ❑ Salário – Salário de um funcionário
 - ❑ Número real
 - ❑ Domínio: $[0.00, 99999999.99]$

N

18

Salário

3437.43



TIPOS DE VARIÁVEIS ESTRUTURADOS

❑ Declaração de uma Ficha:

Nome: texto

Número: número inteiro

Telefone: número

Data nascimento: data

❑ Declaração uma variável do tipo Ficha

F: Ficha

❑ Acesso

F.Nome ← "Paulo Nunes"

F.Telefone ← 271220161

Paulo Nunes
271220161



VECTOR

- ❑ Permite armazenar vários valores.
- ❑ O acesso a cada elemento é efectuado utilizando um endereço (número) entre parênteses recto [número].
- ❑ Notas[10]: Inteiros
 - ❑ vector que permite armazenar 10 valores inteiros
- ❑ Acesso
 - Notas[1] $\leftarrow$ 12
 - Notas[2] $\leftarrow$ 1



VECTOR: EXEMPLOS

Notas[10]: Inteiros

12	16								
----	----	--	--	--	--	--	--	--	--

1 2 3 4 5 6 7 8 9 10

1^a nota

2^a nota

Classificações[4]: Texto

Reprovado	Suficiente	Bom	Muito Bom
-----------	------------	-----	-----------



VECTOR: EXEMPLOS

AnoNascimento (Inteiro T4) [9]

1

--	--	--	--

2

--	--	--	--

3

--	--	--	--

4

--	--	--	--

5

--	--	--	--

6

--	--	--	--

7

--	--	--	--

8

--	--	--	--

9

--	--	--	--

Pesos (Real T6) [6]

1

			.		
--	--	--	---	--	--

2

			.		
--	--	--	---	--	--

3

			.		
--	--	--	---	--	--

4

			.		
--	--	--	---	--	--

5

			.		
--	--	--	---	--	--

6

			.		
--	--	--	---	--	--



VECTOR PARALELOS: EXEMPLOS

Idade (Inteiro T2) [6]

1		
2		
3		
4		
5		
6		

Peso (Real T5) [6]

1			.			
2			.			
3			.			
4			.			
5			.			
6			.			

Altura (Inteiro T3) [6]

1			
2			
3			
4			
5			
6			

Na mesma linha de vetor está informação sobre a mesma pessoa.

Exemplo: linha 1 - Miguel

Idade[1] = 19

Peso[1] = 65.453

Altura[1] = 176



MATRIZ

- ❑ Permite armazenar vários valores organizado em linhas e colunas.

- ❑ O acesso a cada elemento é efectuado utilizando dois endereços (número) entre parênteses recto [linha][coluna].

- ❑ Exemplos

Temperaturas [3][2]: Inteiro

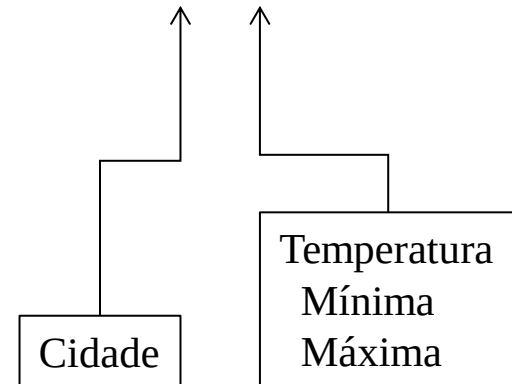
Imagem[1024][1024]: Inteiro



MATRIZ: EXEMPLOS

	1	2	
1	-2	3	Guarda
2	5	9	Viseu
3	8	14	Lisboa
	Mínima	Máxima	

Temperaturas [3][2]: Real



Temperaturas[1][1] = -2

Temperatura mínima para a cidade da guarda



MATRIZ: EXEMPLOS

Alunos

1. Marco
2. Henrique
3. Tiago
4. Edson

Disciplinas

1. Álgebra e Geometria Analítica
2. Algoritmos e Estruturas de Dados
3. Análise Matemática
4. Introdução à Física
5. Introdução à Programação

Aluno		Disciplina			
Notas (Inteiro T2)		[4]	[5]		
1					
2					
3					
4		16			

$$\text{Notas}[4][2] = 16$$

Nota do Edson a AED
na época de frequência.



MATRIZ: EXEMPLOS

Damas (Real T1) [8][8]

1								
2								
3								
4								
5								
6								
7								
8								

Tabuleiro de Damas

Valores:

0 – vazio

B – Peça branca

P – Peça preta



CUBOS: E N DIMENSÕES

- ❑ Permite armazenar vários valores organizado em linha, colunas e altura.
 - ❑ O acesso a cada elemento é efectuado utilizando três endereços (número) entre parênteses recto [linha][coluna][altura].
- ❑ Exemplos
 - Temperaturas [3][2][31]: Inteiro
Duas temperaturas para 3 para 31 dias.
 - Temperaturas [3][2][8] = 14 // Lisboa/máxima/dia 8



3D: EXEMPLOS



Alunos

1. Marco
2. Henrique
3. Tiago
4. Edson

Disciplinas

1. Álgebra e Geometria Analítica
2. Algoritmos e Estruturas de Dados
3. Análise Matemática
4. Introdução à Física
5. Introdução à Programação

Épocas

1. Frequência
2. Exame
3. Recurso

Notas (Inteiro T2) [4][5][3]

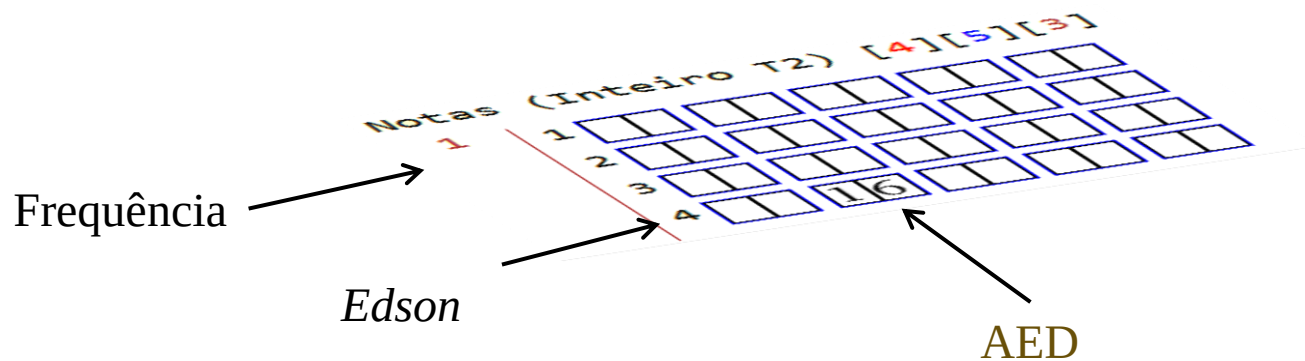
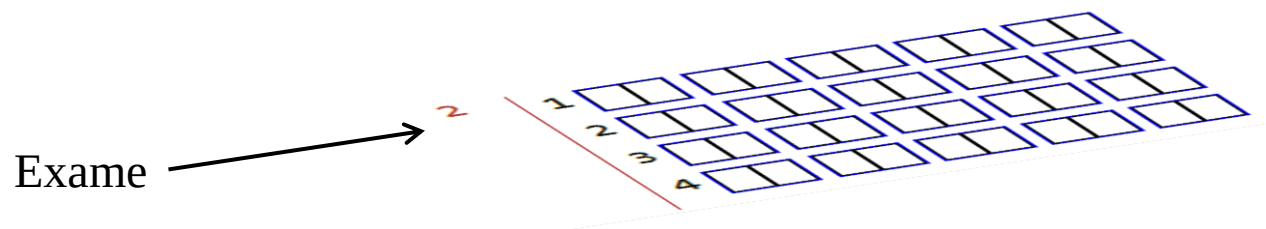
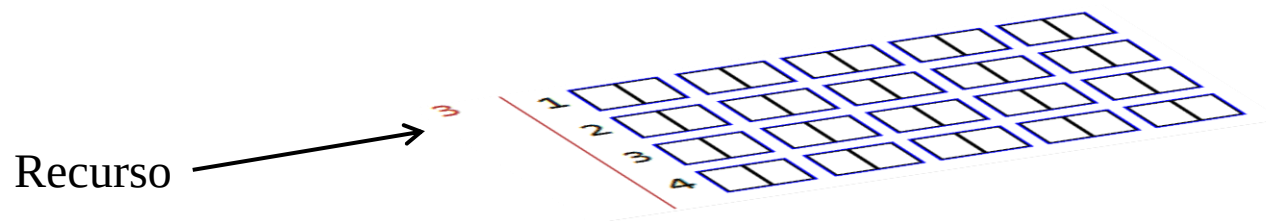
1	1						
	2						
	3						
	4		16				
2	1						
	2						
	3						
	4						
3	1						
	2						
	3						
	4						

Notas[4][2][1] = 16

Nota do Edson a AED
na época de frequência.

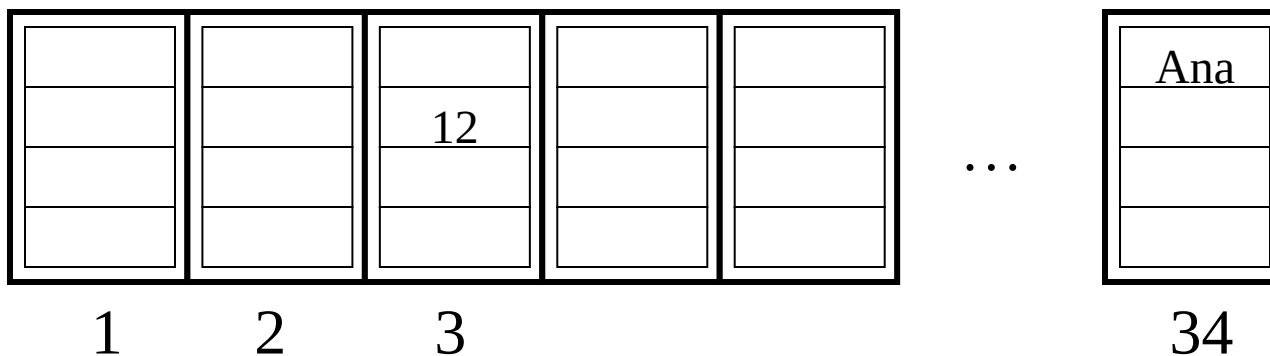


3D: EXEMPLOS



VECTOR DE FICHAS

Alunos[34]: Ficha



Acesso

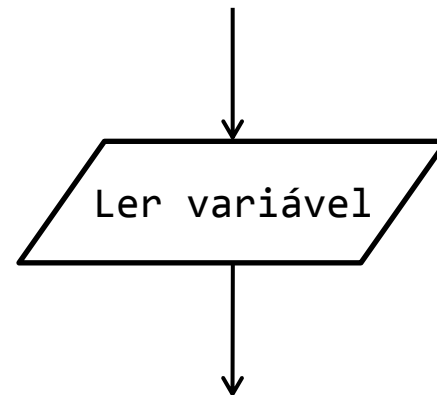
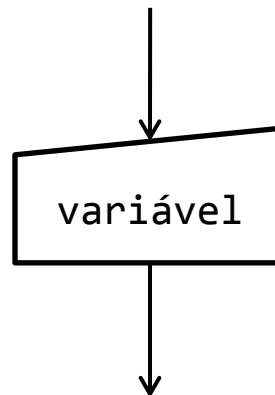
Alunos[3].numero = 12

Alunos[34].nome = "Ana"

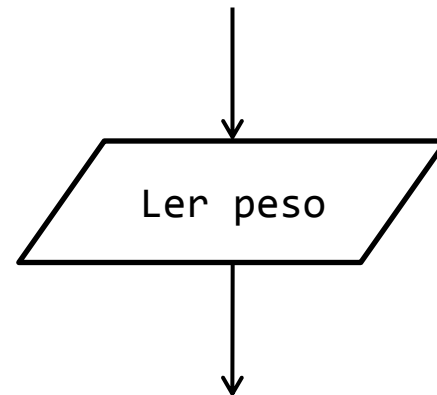
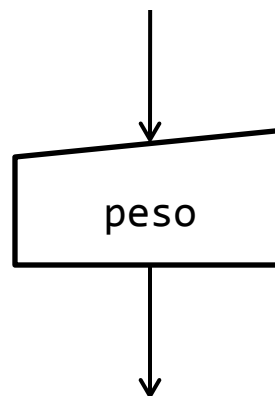


INSTRUÇÕES DE ENTRADA

□ Ler variável



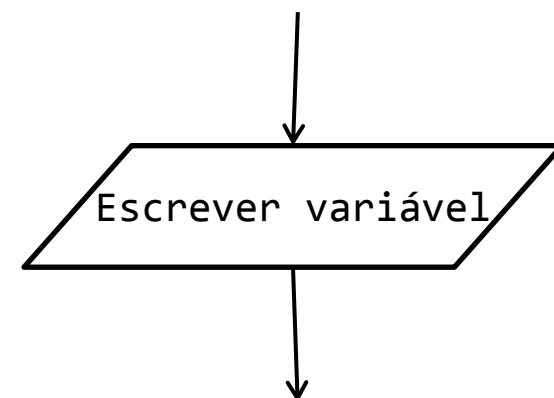
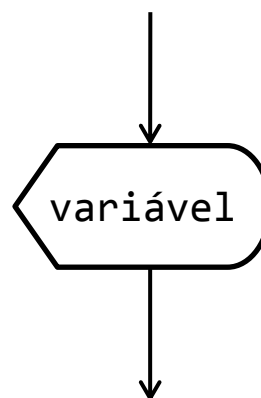
Ler peso



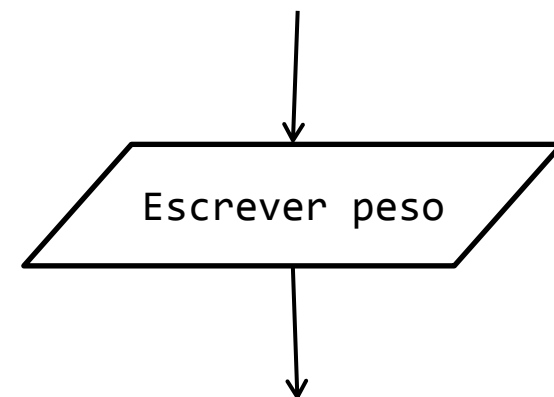
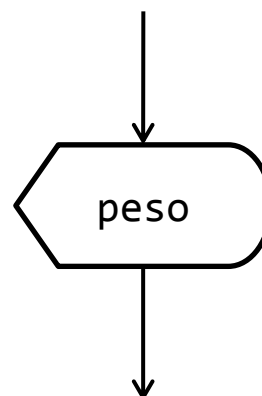


INSTRUÇÕES DE SAÍDA

□ Escrever variável



Escrever peso





ANO BISSEXTO

Início:

Inteiro mes , ano , ultimodia

mes <- 2

ano <- 2000

Caso (mês)

caso 1 , 3 , 5 , 7 , 8 , 10 , 12 :

ultimodia <- 31

caso 4 , 6 , 9 , 11 :

ultimodia <- 30

defeito :

Se $\text{ano} \% 4 = 0$ e $\text{ano} \% 100 \neq 0$ ou $\text{ano} \% 400 = 0$ Então

ultimodia <- 29

Senão

ultimodia <- 28

FimSe

FimCaso

escrever "Último dia : " , ultimodia

Fim.

- ❑ Decisão
 - ❑ Se
 - ❑ Caso
- ❑ Repetitivas
 - ❑ Enquanto
 - ❑ Até
 - ❑ Para

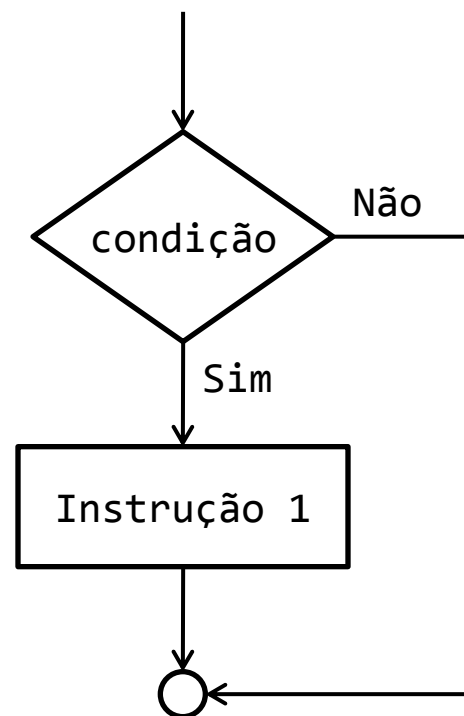


SE: UMA INSTRUÇÃO

Se (condição) então

Instrução 1

FimSe



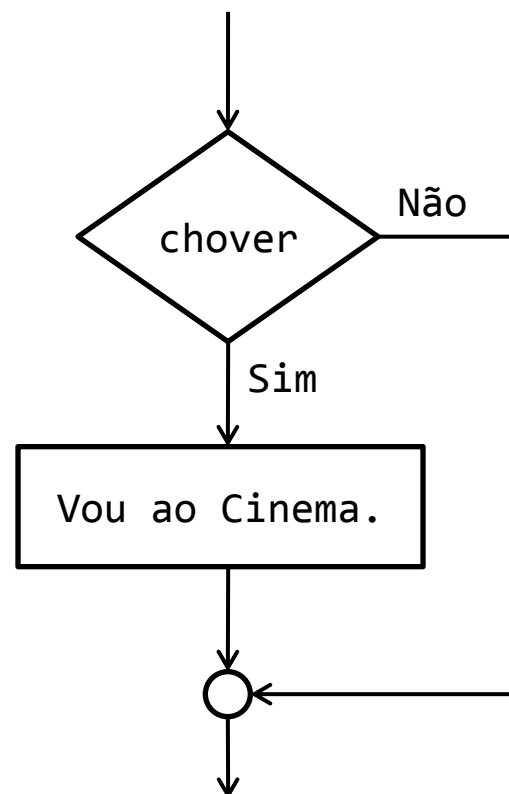


SE: UMA INSTRUÇÃO

Se (chover) então

Vou ao Cinema.

FimSe

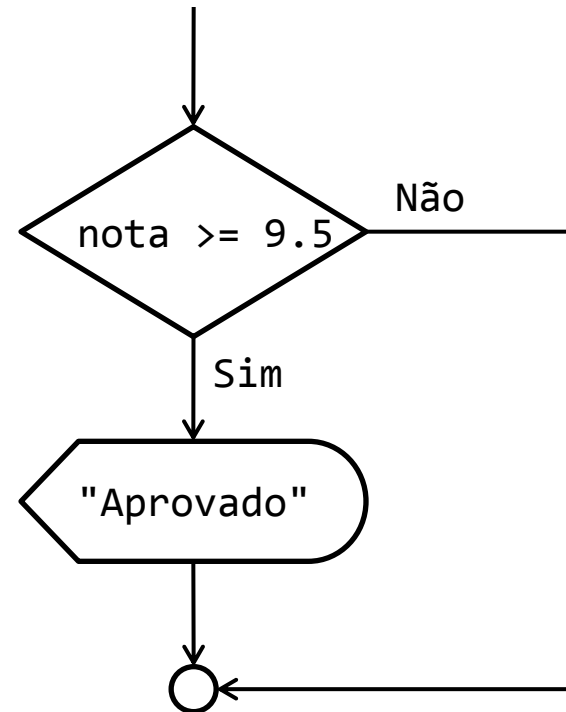


SE: UMA INSTRUÇÃO

Se (nota $\geq$ 9.5) **então**

Escrever "Aprovado"

FimSe



SE: N INSTRUÇÕES

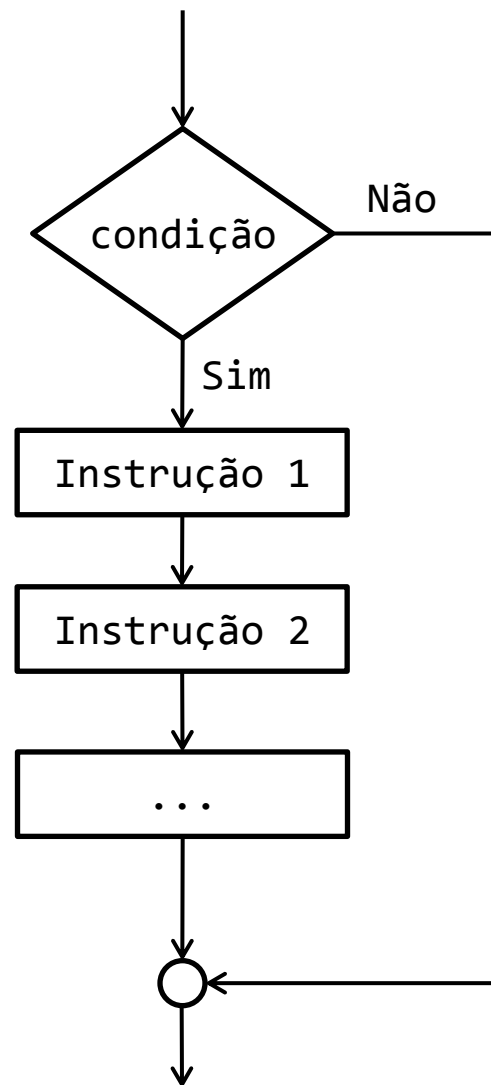
Se (condição) **então**

Instrução 1

Instrução 2

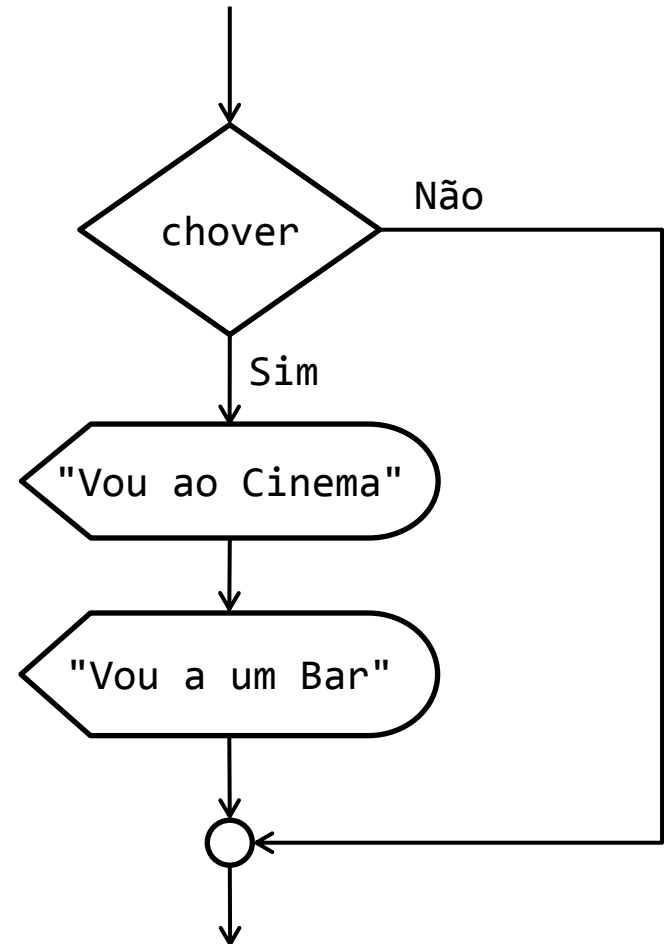
...

FimSe



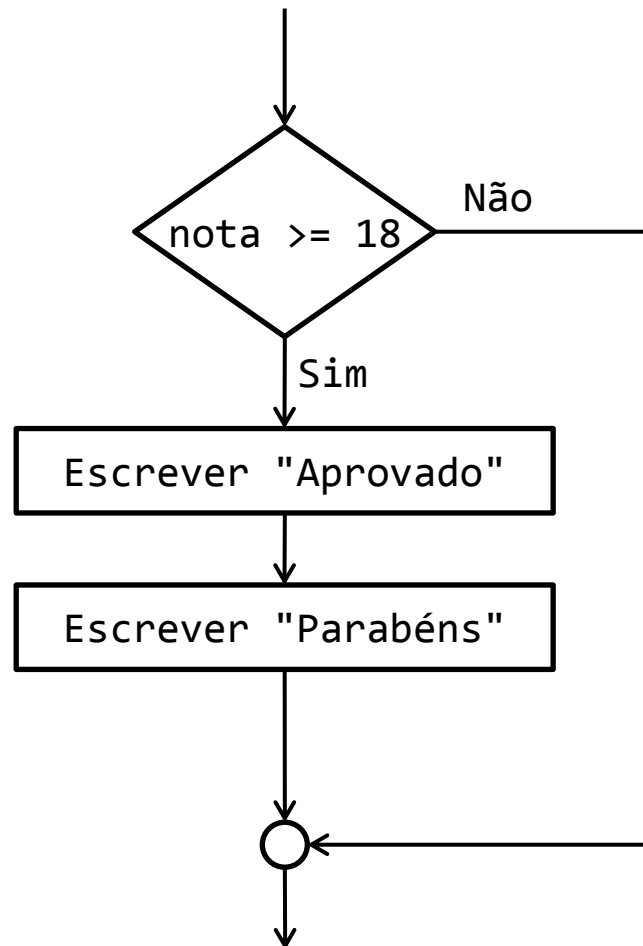
SE: N INSTRUÇÕES

Se (chover) **então**
 Vou ao Cinema.
 Vou a um Bar.
FimSe



SE: N INSTRUÇÕES

Se (nota $\geq$ 18) **então**
 Escrever "Aprovado"
 Escrever "Parabéns"
FimSe





SE, ENTÃO, SENÃO

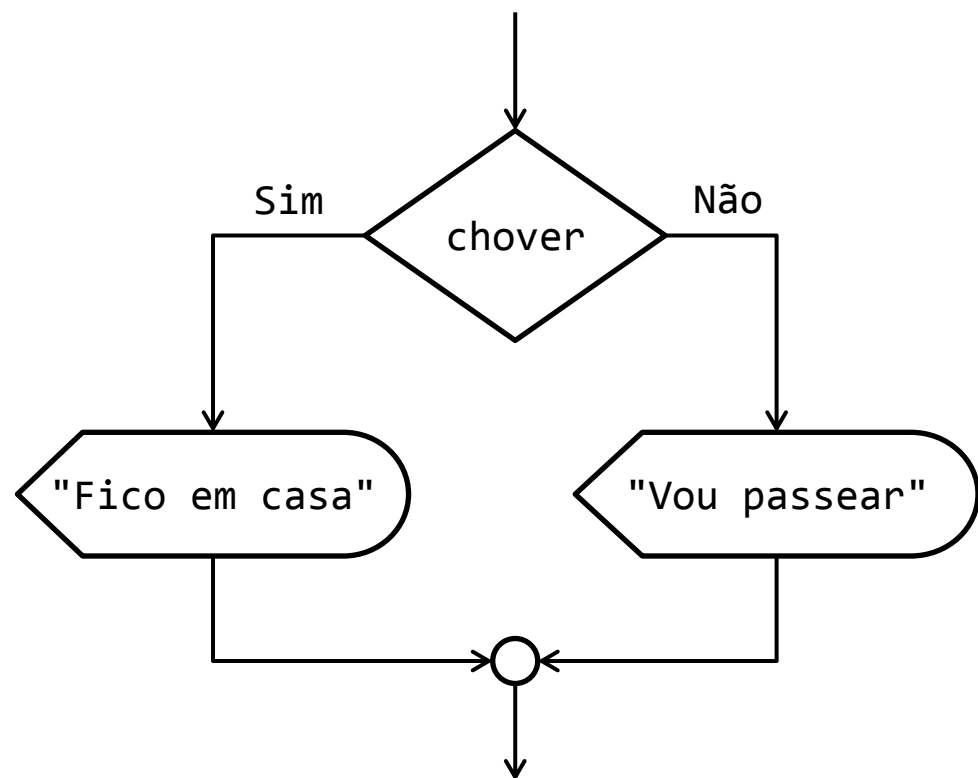
Se (chover) então

Fico em casa.

Senão

Vou passear.

FimSe





SE, ENTÃO, SENÃO

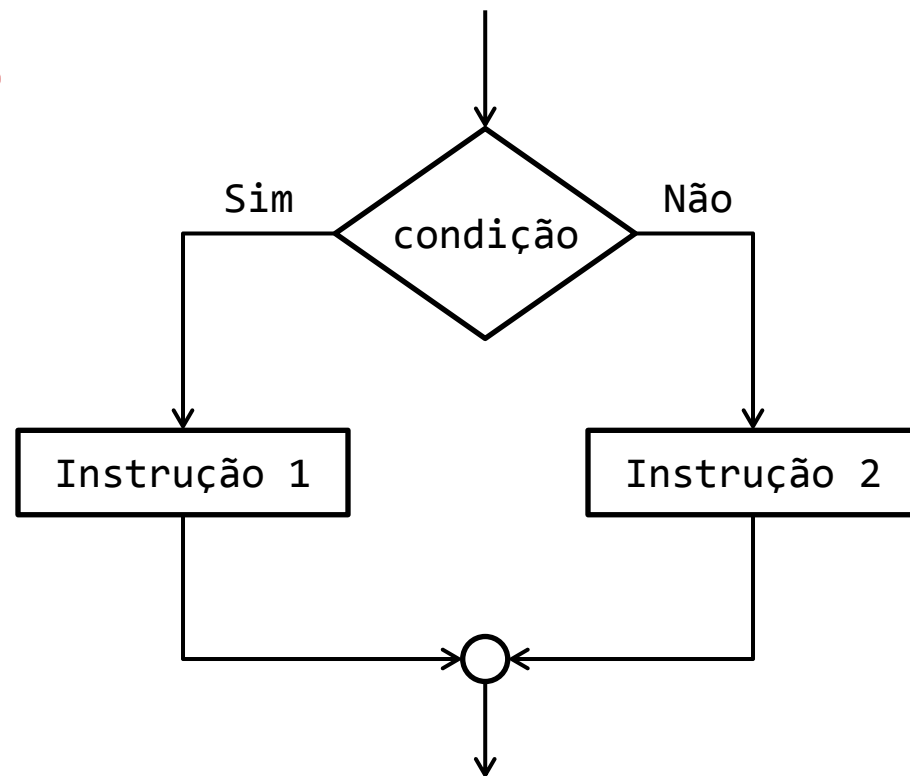
Se (condição) então

Instrução 1

Senão

Instrução 2

FimSe



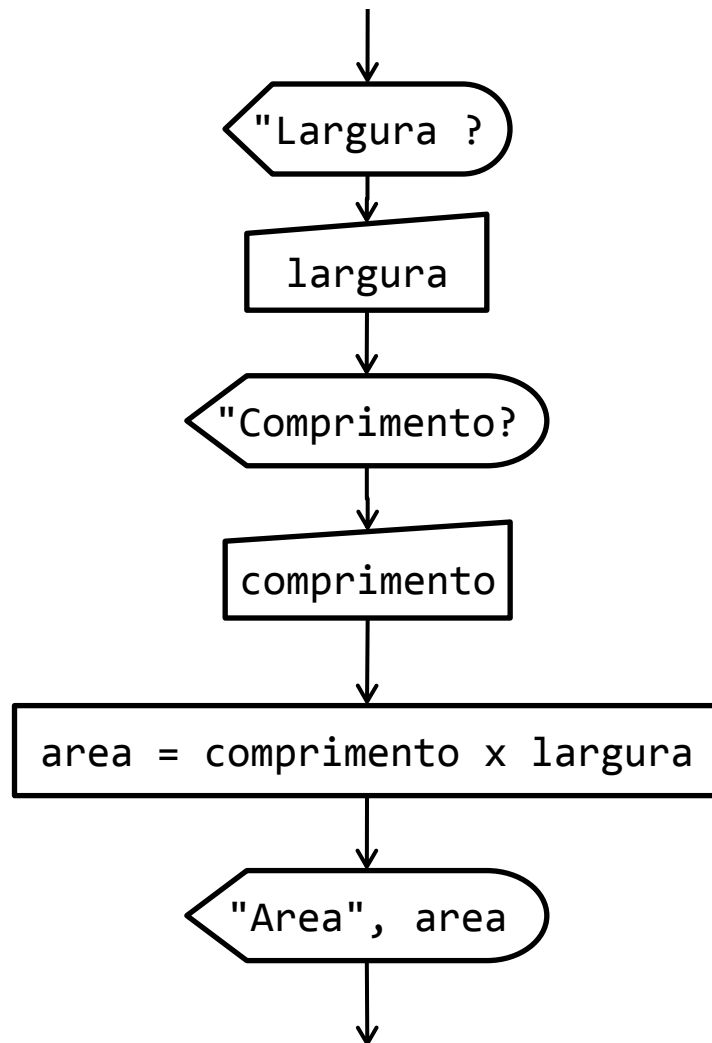


EXEMPLO: ÁREA: PSEUDCÓDIGO

1. Escrever "Largura ?"
2. Ler largura
3. Escrever "Comprimento ?"
4. Ler comprimento.
5. $area = largura \times comprimento$
6. Escrever "Area: ", area



EXEMPLO: ÁREA: FLUXOGRAMA





SE: ENCADEADO

Se (condição1) então

 Instrução1

SeNão

 Se (condição2) então

 Instrução 2

 SeNão

 Instrução 3

 FimSe

FimSe



SE: ENCADEADO

Se (nota ≥ 10) então

Escrever "Aprovado"

SeNão

Se (nota ≥ 8) então

Escrever "Insuficiente."

SeNão

Escrever "Mau!"

FimSe

FimSe

CASO

Caso (variável)

caso <constante1> Fazer

Instruções

caso <constante 2> Fazer

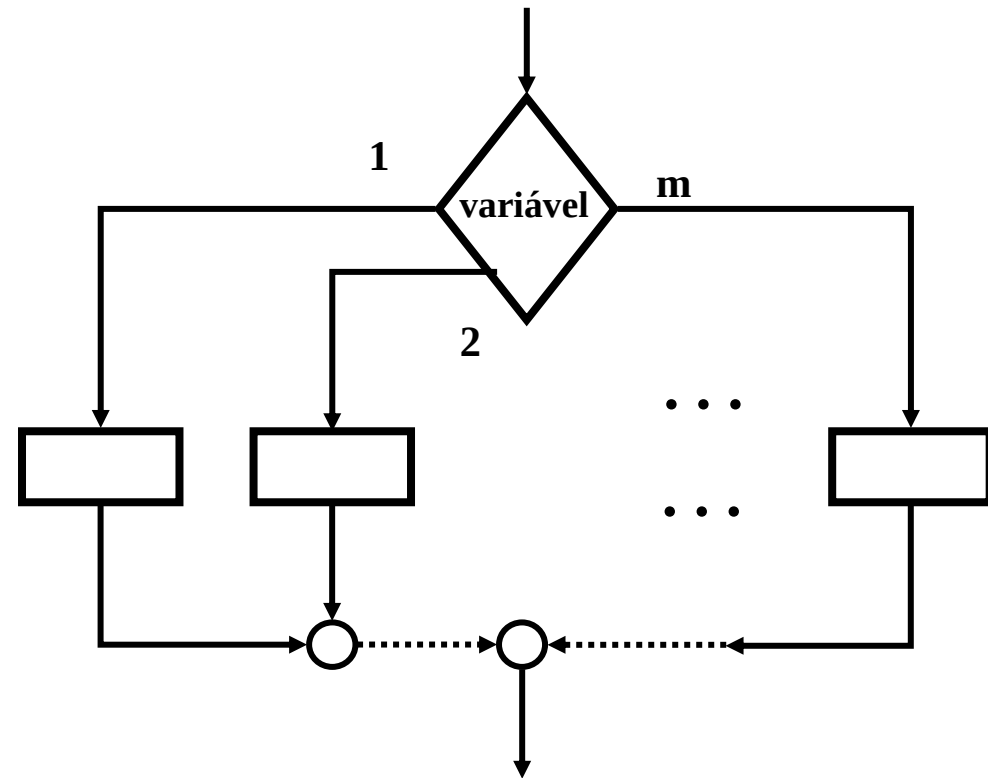
Instruções

...

caso <constante m> Fazer

Instruções

FimCaso



FAZER - ENQUANTO

Fazer

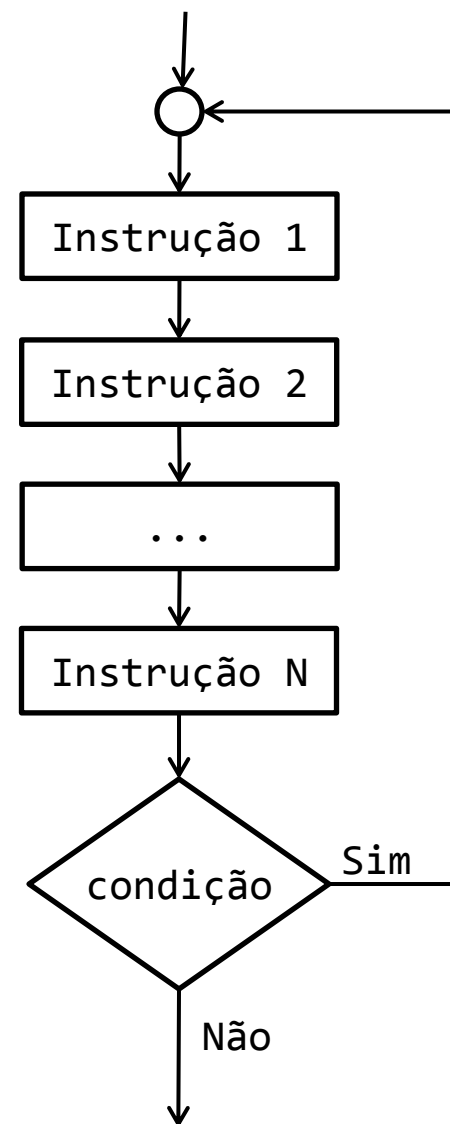
instrução1

instrução2

...

instrução n

Enquanto (condição)



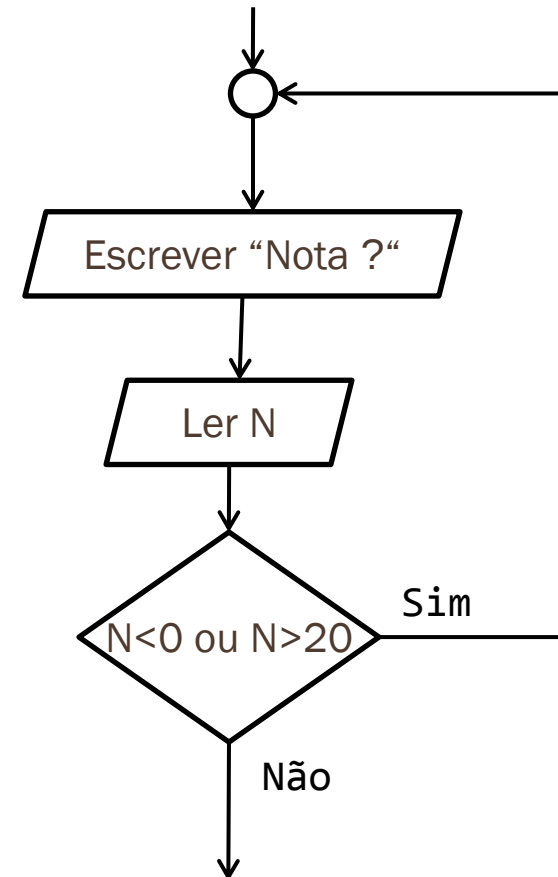
FAZER – ENQUANTO: EXEMPLO

Fazer

Escrever “Nota ?”

Ler N

Enquanto ($N < 0$ ou $N > 20$)





FAZER - ENQUANTO

- ❑ O conjunto de instruções é executado pelo menos uma vez, independentemente da condição ser verdadeira ou falso.
- Faz as instruções enquanto a condição for verdadeira.
- Utilizado na entradas de dados com validação
 - Os dados têm de ser introduzidos pelo menos uma vez.

FAZER - ATÉ

Fazer

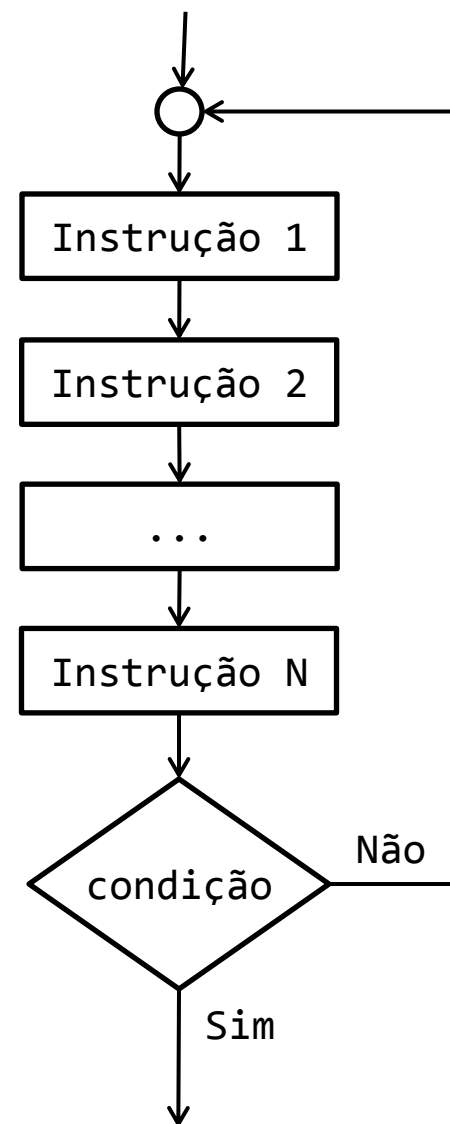
instrução1

instrução2

...

instrução n

Até (condição)





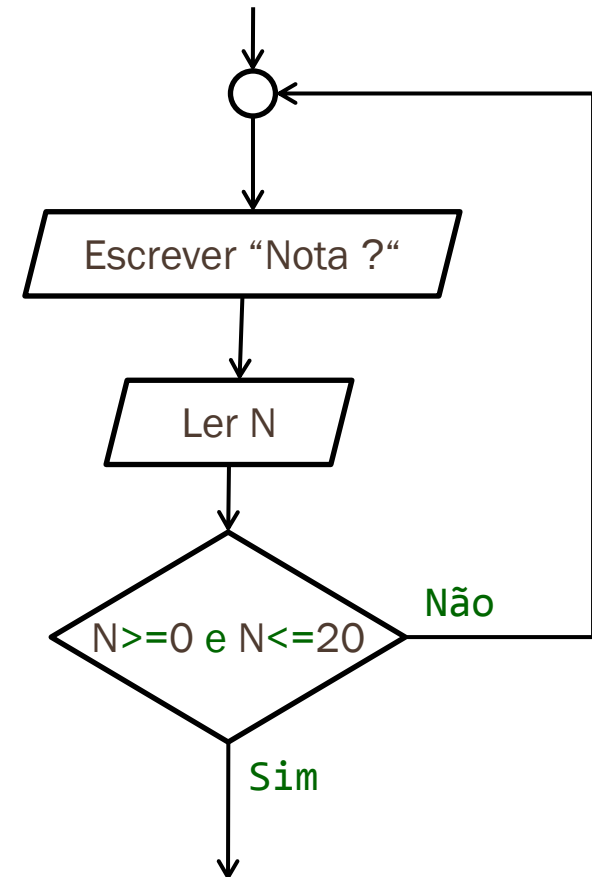
FAZER – ATÉ: EXEMPLO

Fazer

Escrever “Nota ?”

Ler N

Até ($N \geq 0$ e $N \leq 20$)





FAZER – ATÉ / FAZER – ENQUANTO

Fazer

Escrever “Nota ?”

Ler N

Até ($N \geq 0$ e $N \leq 20$)



Repetir até ser válido

Fazer

Escrever “Nota ?”

Ler N

Enquanto ($N < 0$ ou $N > 20$)

Repetir enquanto for inválido



FAZER - ATÉ

- ❑ O conjunto de instruções é executado pelo menos uma vez, independentemente da condição ser verdadeira ou falso.
- Faz as instruções até a condição ser verdadeira.
- Utilizado na entradas de dados com validação
- ❑ É equivalente ao Fazer – Enquanto com a condição na negativa.

ENQUANTO - FAZER

Enquanto (condição) **Fazer**

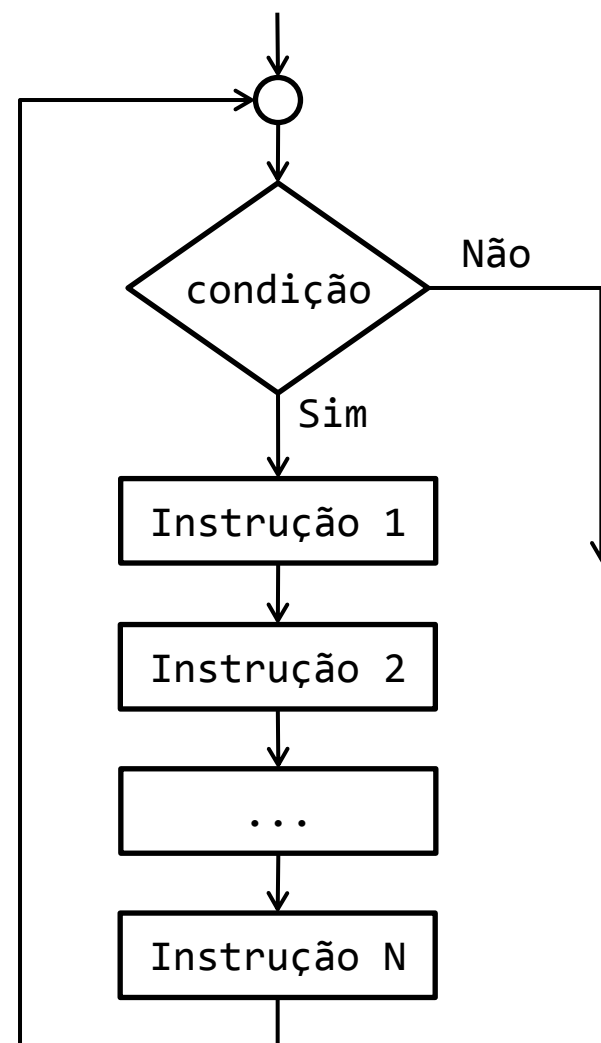
Instrução1

Instrução2

...

Instrução M

FimEnquanto





ENQUANTO - FAZER

- ❑ O conjunto de instruções pode ser executado zero ou mais vezes:
 - ❑ nenhuma vez:
 - ❑ quando a condição é falsa na primeiro teste da condição.
 - ❑ uma ou mais vezes:
 - ❑ Enquanto a condição for verdadeira.



PARA - ATÉ

Para variável = início até
fim [**passo** incremento]

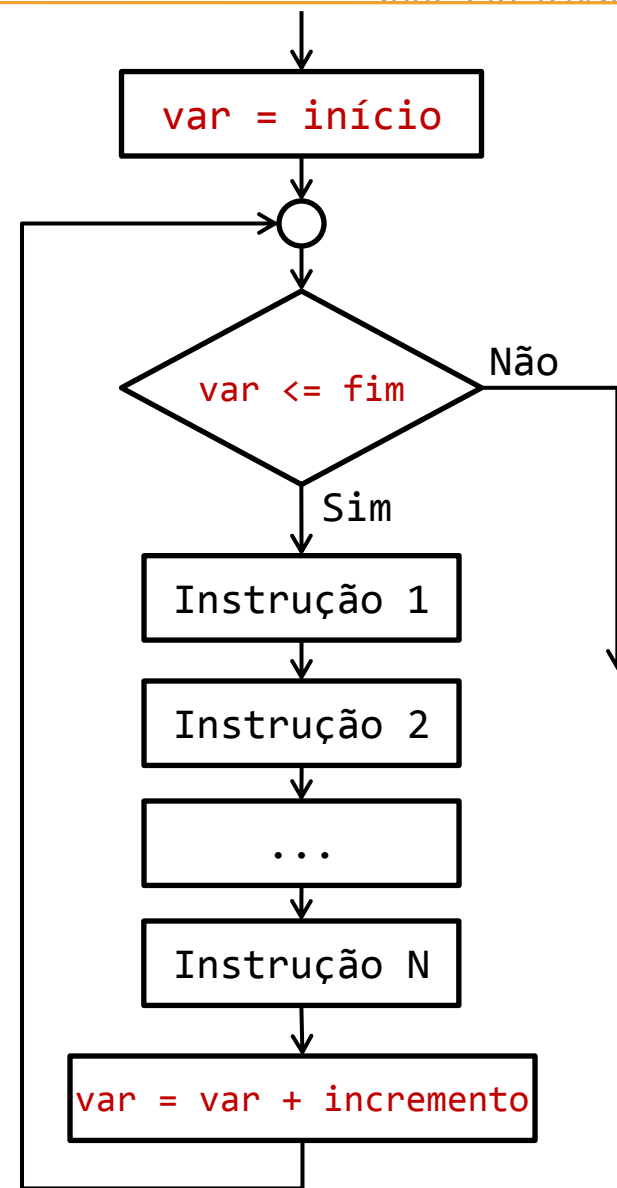
Instrução1

Instrução2

...

Instrução N

FimPara





PARA - ATÉ

- A variável (var) da condição é inicializada no início.
- O conjunto de instruções é executado enquanto o valor inicial for menor ou igual ao valor final.
- Ao fim de cada ciclo é incrementado o valor da variável de uma unidade.



PARA - ATÉ

Início:

$S=0$

Para $c = 1$ **até** 3 **passo** 1

Escrever “Nota ?”

Ler Nota

$S \leftarrow S + \text{Nota}$

FimPara

$M = S / 3$

Escrever “Média”, M

Fim.

Início:

$S=0$

Escrever “Quantas notas ?”

Ler N

Para $c = 1$ **até** N **passo** 1

Escrever “Nota ?”

Ler Nota

$S \leftarrow S + \text{Nota}$

FimPara

$M = S / N$

Escrever “Média”, M

Fim.



PARA – ATÉ: TABULADA

Início:

Escrever “Tabuada ?”

Ler N

Para i = 1 **até** 10 **passo** 1

Escrever N, " x ", i, " = ", N x i

FimPara

Fim.



PARA – ATÉ: FAZER - ENQUANTO

Início:

S=0

Para c = 1 **até** 3 **passo** 1

Fazer

Escrever “Nota ?”

Ler Nota

Enquanto (Nota<0 ou Nota>20)

S ← S + Nota

FimPara

M = S / 3

Escrever “Média“, M

Fim.

Início:

S=0

Fazer

Escrever “Quantas notas ?”

Ler N

Enquanto (N < 0)

Para c = 1 **até** N **passo** 1

Fazer

Escrever “Nota ?”

Ler Nota

Enquanto (Nota<0 ou Nota>20)

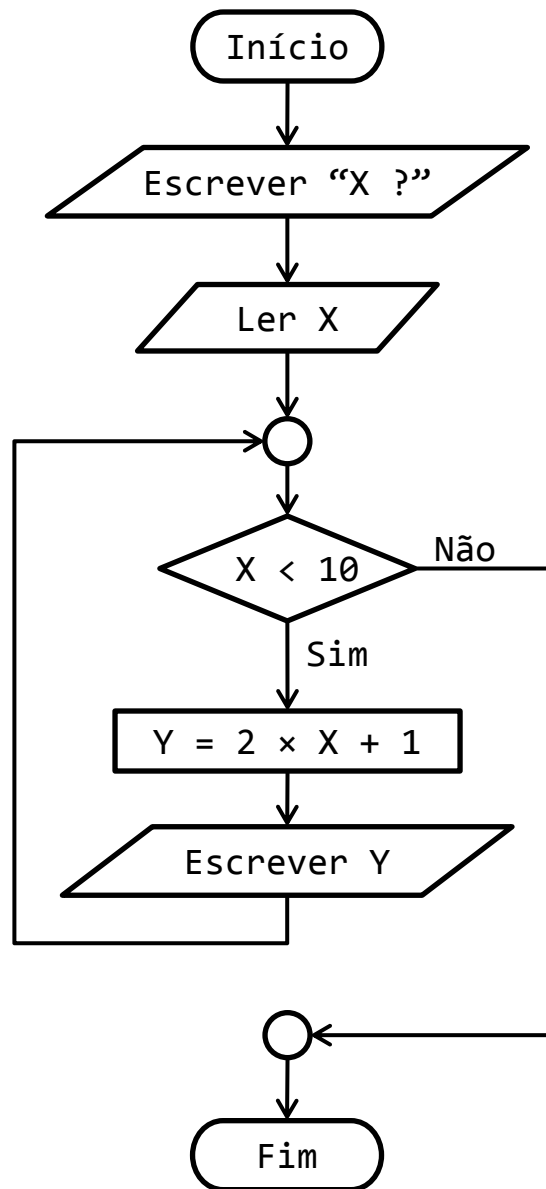
S ← S + Nota

FimPara

M = S / N

Escrever “Média“, M

Fim.





ALGORITMOS - ESTRUTURAS DADOS

- ❑ Algoritmos são métodos para resolver problemas:
 - ❑ problemas têm dados;
 - ❑ dados são tratados computacionalmente.



ALGORITMOS - ESTRUTURAS DADOS

- ❑ O Processo de organização dos dados pode determinar a eficiência do algoritmo:
 - ❑ algoritmos simples podem requerer estruturas de dados complexas
 - ❑ algoritmos complexos podem requerer estruturas de dados simples



ALGORITMOS - ESTRUTURAS DADOS

- ❑ Para maximizar a eficiência de um algoritmo as estruturas de dados que se usam têm de ser projectadas em simultâneo com o desenvolvimento do algoritmo.



PORQUÊ ESTUDAR ALGORITMOS?

- ❑ Quanto se utilizam meios computacionais:
 - ❑ queremos sempre que a computação seja mais rápida;
 - ❑ queremos ter a possibilidade de processar mais dados;
 - ❑ queremos algo que seria impossível sem o auxílio de um computador.



DOCUMENTAÇÃO

- ❑ Algoritmo:
- ❑ Objectivo:
- ❑ Variáveis:
 - ❑ Entrada:
 - ❑ Saída:
 - ❑ Auxiliares:
- ❑ Desenvolvido por:
- ❑ Data:
- ❑ Versão:
- ❑ Obs:



DOCUMENTAÇÃO: EXEMPLO

Algoritmo: DoisNumeros

Objectivo: Permite ler dois números reais e escrever o resultado da sua soma, diferença, produto e divisão formatados com 8 dígitos significativos, sendo reservados 3 para a parte fraccionária.

Variáveis:

Entrada:

primeiro (real) – primeiro número
segundo (real) – segundo número

Saída:

soma (real) – soma dos dois números
diferença (real) – diferença dos dois números
produto (real) – produto dos dois números
divisão (real) – divisão dos dois números

Auxiliares:

Desenvolvido por: Prof. Paulo Nunes

Data: Setembro/2004

Versão: 1.0

Obs:



DOCUMENTAÇÃO: ALGORITMO

Início:

Escrever “Diga o primeiro número real: “ // entrada de dados

Ler primeiro

Escrever “Diga o segundo número real: “

Ler segundo

soma = primeiro + segundo // processamento

diferença = primeiro - segundo

produto = primeiro $\times$ segundo

divisão = primeiro / segundo

Escrever soma // saída de dados

Escrever diferença

Escrever produto

Escrever divisão

Fim.



DOCUMENTAÇÃO

Objectivo: Permite ler três números inteiros e escrever a sua soma, média e média arredondada para o inteiro mais próximo.

Variáveis:

Entrada: primeiro (inteiro) - primeiro número
segundo (inteiro) - segundo número
terceiro (inteiro) - terceiro número

Saída: média (real) - média dos três números
média_arredondada (real) - média arredondada inteiro + próximo

Auxiliares:

Desenvolvido por: Prof. Paulo Nunes

Data: Setembro/2004

Release: 1.0

Versão: A

Obs:



ÁREA TRIÂNGULO

Algoritmo: AreaTriangulo

Objectivo: Permite calcular a área de um triângulo.

Variáveis:

Entrada: base (real)- base do triângulo

altura (real)- altura do triângulo

Saída: A (real) – área do triângulo

Auxiliares:

Desenvolvido por: Prof. Paulo Nunes

Data: Setembro/2004

Release: 1.0

Versão: A

Obs:

Início:

// entrada de dados

Escrever “Cálculo da área de um triângulo”

Escrever “Diga base ?”

Ler base

Escrever “diga altura ?”

Ler altura

// processamento

$area = (base \times altura) / 2$

// saída de dados

Escrever “Área: “, area

Fim.



ALGORITMOS DE PESQUISA E ORDENAÇÃO

- ❑ Pesquisa
 - ❑ Sequencial
 - ❑ Binária
- ❑ Ordenação
 - ❑ Selecção
 - ❑ Inserção
 - ❑ Bubble sort
 - ❑ QuickSort



BIBLIOGRAFIA

- ❑ <http://www.dei.estt.ipt.pt/portugol/book/export/html/6>

